vLLM Neuron Beta (Enhanced Plugin)

version 0.19.0.0

AWS Neuron

May 26, 2026

Contents

vLLM Neuron Beta (Enhanced Plugin) **1**

How to set up vLLM on Neuron 1

Task overview 1

Related information 1

Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron 1

Prerequisites 2

Step 1: Launch the API server 2

Step 2: Verify the endpoint with `curl` 2

Step 3: Call the API with the OpenAI Python SDK 3

Confirmation 4

Common issues 4

Clean up 4

Next steps 4

Further reading 4

Quickstart: Run offline inference with vLLM on Neuron 4

Prerequisites 4

Step 1: Run a batch inference job 5

Step 2: Try advanced configuration options (optional) 5

Common issues 6

Clean up 6

Next steps 6

Further reading 6

How to use vLLM on Neuron features in your deployment 6

Task overview 6

Prerequisites 7

Instructions 7

Confirm your work 11

Common issues 12

Related information 12

Reference: vLLM on Neuron feature and model compatibility matrix 12

Applies to 13

Overview 13

Usage 13

Feature support by model 13

Notes on conditional support 14

Related reference 14

Learn more 14

How to onboard a model to vLLM on Neuron 14

Task overview 14

Prerequisites 15

End-to-end onboarding flow 15

Instructions 15

1: Implement the model 15

2: Register the model with vLLM 25

3: Compile and run a smoke test 25
4: Validate accuracy 26
5: Benchmark and tune performance 27
Confirm your work 27
Common issues 27
Related information 28
How to debug accuracy issues with vLLM on Neuron 28
Task overview 28
Prerequisites 28
Validation levels 29
Debugging instructions 29
Step 1: Rule out sampling variance 29
Step 2: Reproduce the issue with a minimal, deterministic prompt 30
Step 3: Compare outputs against a reference 30
Step 4: Isolate the introduction point with a layered check 30
Step 5: Interpret the divergence pattern 31
Step 6: Check for common causes 31
Wrong checkpoint format or incomplete weight loading 31
Dtype mismatch between reference and deployment 32
Quantization-induced drift exceeding tolerance 32
KV cache inconsistency under prefix caching or paged attention 32
Speculative decoding verification failure 32
Unsupported feature combination 32
Step 7: Apply a fix and validate 32
Confirm your work 33
Common issues 33
Tokenizer mismatch between reference and deployment 33
Quantization-induced drift exceeds quality threshold 33
Prefix caching produces different output than non-cached runs 33
Output looks correct for short prompts but degrades at longer context 33
Accuracy differs between tensor parallelism degrees 33
Accuracy tools reference 33
Logit validation 34
KV cache analysis 34
Tensor capture 35
Tensor compare 36
Module-level testing 37
Related information 37
How to deploy gpt-oss with vLLM on Neuron 37
Task overview 37
Tested versions 37
Target configuration 38
Prerequisites 38
Instructions 39
Confirm your work 40
Common issues 41

Related information 41
Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron 41
Overview 41
Before you start 42
Model details 42
Prerequisites 42
Prepare your environment 42
Step 1: Run the non-speculative baseline 43
Step 2: Run EAGLE3 and compare 43
Step 3: Tune for your workload 45
Confirmation 45
Common issues 45
Clean up 45
Next steps 46
Tutorial: Configure disaggregated inference with 1P1D and xPyD on vLLM on Neuron 46
Overview 46
Before you start 46
Prerequisites 46
Prepare your environment 47
Step 1: Launch the prefill server 47
Step 2: Launch the decode server 48
Step 3: Launch the proxy server 48
Step 4: Validate the 1P1D deployment 49
Step 5: Scale to xPyD (multi-node) 49
Confirmation 50
Common issues 50
Clean up 51
Next steps 51

vLLM Neuron Beta (Enhanced Plugin)

vLLM on Neuron is the recommended path for production LLM serving on AWS Trainium and Inferentia starting with Neuron 2.30. This section covers the end-to-end developer experience for the vLLM-Neuron plugin: setup, quickstarts for online and offline serving, user and feature guides, model recipes, tutorials for advanced deployments, and reference material.

For guidance on choosing between vLLM on Neuron and the legacy NxD Inference library, see **`/inference/about-inference`**.

How to set up vLLM on Neuron

Task overview

This topic discusses how to install and configure the vLLM-Neuron plugin on AWS Trainium or Inferentia so you can serve large language models with vLLM on Neuron.

> **Note**
>
> For detailed setup and installation instructions, refer to the provided beta setup guide included with this documentation package.

Related information

- Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron — Launch an OpenAI-compatible API server with vLLM on Neuron.
- Quickstart: Run offline inference with vLLM on Neuron — Run offline batch inference with vLLM on Neuron.
- How to use vLLM on Neuron features in your deployment — Detailed coverage of vLLM on Neuron features and configuration.

Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron

This quickstart walks you through launching a vLLM server on AWS Neuron that exposes an OpenAI-compatible chat completions API. You will start the server, send a request with `curl`, and then call it from the OpenAI Python SDK. When you have completed it, you will have a running inference endpoint you can integrate into any application that speaks the OpenAI API.

This quickstart is for: Developers who have completed the **`setup guide`** and want the fastest path to an interactive serving endpoint on Neuron

Time to complete: ~20 minutes

Prerequisites

Before you begin, make sure you have:

- A **Trn2** EC2 instance with network access to Hugging Face Hub.
- The vLLM-Neuron plugin installed and verified. Refer to the provided beta setup guide for installation instructions.
- Access to the `meta-llama/Llama-3.1-8B-Instruct` model on Hugging Face (request access at the model card if needed).

Step 1: Launch the API server

In this step, you start an OpenAI-compatible endpoint serving Llama 3.1 8B Instruct.

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
  --tensor-parallel-size 8 \
  --max-model-len 4096 \
  --max-num-seqs 4 \
  --port 8000 \
  --additional-config '{
    "neuron_config": {
      "on_device_sampling_config": {"all_greedy": "true"}
    }
  }'
```

What each argument does:

- `--tensor-parallel-size 8` — shards the model across 8 NeuronCores (fits an 8B model comfortably on Trn2).
- `--max-model-len 4096` — maximum sequence length (prompt + generation) the server will accept.
- `--max-num-seqs 4` — maximum concurrent requests batched together.
- `--additional-config` — passes Neuron-specific configs; here enabling greedy on-device sampling.

The first launch compiles the model for Neuron, which takes several minutes. Once ready you will see:

```
INFO:     Started server process [12345]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Step 2: Verify the endpoint with `curl`

In this step, you confirm the server is healthy and responding to chat completion requests.

First, check the health endpoint:

```
curl http://localhost:8000/health
```

You should receive an empty `200 OK` response. Next, send a chat completion request:

```
curl -s http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
        "model": "meta-llama/Llama-3.1-8B-Instruct",
        "messages": [
          {"role": "user",
           "content": "What is the capital of France? Reply in one sentence."}
        ],
        "max_tokens": 30,
        "temperature": 0
      }' | python -m json.tool
```

Expected output (generation text will vary):


```
{
    "id": "chatcmpl-abc123",
    "object": "chat.completion",
    "created": 1715000000,
    "model": "meta-llama/Llama-3.1-8B-Instruct",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "The capital of France is Paris."
            },
            "finish_reason": "stop"
        }
    ],
    "usage": {
        "prompt_tokens": 18,
        "completion_tokens": 7,
        "total_tokens": 25
    }
}
```


If you see a JSON response with a `choices` array containing generated text, the server is working correctly.

Step 3: Call the API with the OpenAI Python SDK

In this step, you confirm that standard OpenAI client libraries work unchanged against the Neuron endpoint.

Install the SDK if you haven't already:

```
pip install openai
```

Then run the following script:

```
from openai import OpenAI

client = OpenAI(
    api_key="EMPTY",
    base_url="http://localhost:8000/v1",
)

response = client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "user",
         "content": "Explain what Neuron cores are in two sentences."}
    ],
    max_tokens=60,
    temperature=0,
)

print(response.choices[0].message.content)
```

Expected output:

```
Neuron cores are custom-designed machine learning accelerators built by AWS,
optimized for high-throughput inference and training workloads. They are the
processing units inside AWS Inferentia and Trainium chips.
```

If the script prints a coherent completion, you have confirmed end-to-end OpenAI SDK compatibility.

Confirmation

You have launched a vLLM server on Neuron, validated it over HTTP with `curl`, and called it from the OpenAI Python SDK. Any application or framework that targets the OpenAI chat completions API can now point to `http://<your-instance-ip>:8000/v1` and use Neuron-accelerated inference without code changes.

Common issues

- **Server exits during compilation**: Confirm `--tensor-parallel-size` does not exceed the number of available NeuronCores. Run `neuron-ls` to check device count.
- **Requests return 5xx errors after startup**: Lower `--max-num-seqs` or `--max-model-len` if the model runs out of device memory.
- **Hugging Face model download fails**: Verify your `HF_TOKEN` environment variable is set and the token has access to the gated model.

Clean up

1. Stop the API server by pressing `Ctrl+C` in the terminal where it is running.
2. Deactivate your Python environment:

   ```
   deactivate
   ```

3. If you are done testing, stop or terminate your EC2 instance to avoid ongoing charges.

Next steps

- Run batch inference without a server: Quickstart: Run offline inference with vLLM on Neuron
- Explore prefix caching, speculative decoding, quantization, and other features: **`vllm-neuron-feature-guide`**
- Deploy a production-ready model configuration: **`vllm-neuron-model-recipes`**

Further reading

- **`vllm-neuron-user-guide-v1`** — Complete vLLM on Neuron integration reference.
- OpenAI Python SDK documentation — Reference for the client library used in Step 4.

Quickstart: Run offline inference with vLLM on Neuron

This quickstart walks you through running vLLM in offline (batch) inference mode on AWS Neuron. You install the `vllm-neuron` plugin and generate text for a batch of prompts.

This quickstart is for: Developers who want to run offline/batch inference on Neuron without an API server **Time to complete**: ~20 minutes

Prerequisites

Before you begin, make sure you have:

- A **Trn2** EC2 instance with network access to Hugging Face Hub.
- The vLLM-Neuron plugin installed and verified. Refer to the provided beta setup guide for installation instructions.
- Access to the `meta-llama/Llama-3.1-8B-Instruct` model on Hugging Face (request access at the model card if needed).

Step 1: Run a batch inference job

In this step, you run a short Python script that generates completions for three prompts using the Llama 3.1 8B Instruct model.

```
from vllm import LLM, SamplingParams

if __name__ == '__main__':
    llm = LLM(
        model="meta-llama/Llama-3.1-8B-Instruct",
        tensor_parallel_size=8,
        max_num_seqs=1,
        max_model_len=4096,
        additional_config={
            "neuron_config": {
                "on_device_sampling_config": {"all_greedy": "true"},
            },
        },
    )

    prompts = [
        "Hello, my name is",
        "The capital of France is",
        "The future of AI is",
    ]

    outputs = llm.generate(prompts, SamplingParams(top_k=10))
    for output in outputs:
        prompt = output.prompt
        generated_text = output.outputs[0].text
        print(f"Prompt: {prompt!r}")
        print(f"Generated: {generated_text!r}")
```

If the script succeeds, you will see each prompt followed by generated text in the console.

Step 2: Try advanced configuration options (optional)

In this step, you explore optional tuning features that can improve throughput for specific workloads.

Use Eagle speculative decoding when you have an EAGLE checkpoint available:

Below is an example of how to run vLLM inference with an EAGLE3 checkpoint:

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    tensor_parallel_size=8,
    max_model_len=4096,
    max_num_seqs=1,
    speculative_config={
        "method": "eagle3",
        "model": "RedHatAI/Llama-3.1-8B-Instruct-speculator.eagle3",
        "num_speculative_tokens": 3,
    },
    additional_config={
        "neuron_config": {
            "on_device_sampling_config": {
                "all_greedy": "true",
            }
        }
    },
)
```

```
prompts = [
    "Hello, my name is",
    "The capital of France is",
    "The future of AI is",
]

outputs = llm.generate(prompts, SamplingParams(top_k=10))
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}")
    print(f"Generated: {generated_text!r}")
```

Common issues

- **Server exits during compilation**: Confirm `tensor_parallel_size` does not exceed the number of available NeuronCores. Run `neuron-ls` to check device count.
- **Model loading is slow on every run**: Ensure the model weights are cached locally to avoid repeated downloads from Hugging Face Hub.
- **Hugging Face model download fails**: Verify your `HF_TOKEN` environment variable is set and the token has access to the gated model.

Clean up

1. Deactivate your Python environment:

   ```
   deactivate
   ```

2. If you are done testing, stop or terminate your EC2 instance to avoid ongoing charges.

Next steps

- Launch an OpenAI-compatible API server: Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron
- Explore prefix caching, speculative decoding, quantization, and other features: **`vllm-neuron-feature-guide`**
- Deploy a production-ready model configuration: **`vllm-neuron-model-recipes`**

Further reading

- **`vllm-neuron-user-guide-v1`** — Complete vLLM on Neuron integration reference.
- Downloading models from Hugging Face — Instructions for obtaining model checkpoints.

How to use vLLM on Neuron features in your deployment

Task overview

This topic discusses the features available in vLLM on Neuron and how to configure them for your deployment. Use this guide once you have a working environment from the quickstart and need to enable production features such as continuous batching tuning, speculative decoding, quantization, parallelism, disaggregated inference, and structured outputs.

For supported model and feature combinations, see Reference: vLLM on Neuron feature and model compatibility matrix. Where vLLM on Neuron behaves differently from upstream vLLM, the differences are called out inline.

Prerequisites

- **vLLM on Neuron environment:** A working vLLM on Neuron setup. See How to set up vLLM on Neuron.
- **A first request:** You have served at least one request through the quickstart (Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron or Quickstart: Run offline inference with vLLM on Neuron).
- **Compatibility awareness:** See Reference: vLLM on Neuron feature and model compatibility matrix for which features apply to which models.

Most features below use standard vLLM CLI flags. Neuron-specific options are passed as JSON under `--additional-config`, for example: `--additional-config '{"neuron_config": {"ep_degree": 2}}'`. For installation and environment setup, see How to set up vLLM on Neuron.

Instructions

1: Configure continuous batching

- **What it does:** Maintains a queue of in-flight requests and processes many in a single decode batch, chunking long prefills into smaller iterations.
- **When to use:** On by default. Tune only when you need to match a specific traffic profile.
- **How to configure:**
  - `--max-num-seqs` — Upper bound on concurrent requests in a decode batch.
  - `--max-num-batched-tokens` — Maximum tokens per prefill iteration. Longer prompts are chunked into multiple iterations.
  - `num_seqs_buckets` (under `neuron_config`) — Compiled decode batch sizes. Must be ascending; last entry must equal `--max-num-seqs`. Defaults to power-of-2 buckets.
  - `num_batched_tokens_buckets` (under `neuron_config`) — Compiled prefill chunk sizes. Must be ascending; last entry must equal `min(--max-num-batched-tokens, --max-model-len)`. Defaults to power-of-2 buckets.

```
vllm serve meta-llama/Llama-3.3-70B-Instruct \
    --tensor-parallel-size 64 \
    --max-num-seqs 16 \
    --max-num-batched-tokens 8192 \
    --additional-config \
        '{"neuron_config":
        {"num_seqs_buckets": [4, 8, 16],
        "num_batched_tokens_buckets":
        [2048, 4096, 8192]
        }}'
```

- **Trade-offs:** Larger `--max-num-seqs` increases throughput at the cost of decode latency. Larger `--max-num-batched-tokens` reduces prefill iterations for long prompts but uses more memory per iteration. Each bucket compiles a separate NEFF, so more buckets means longer warmup but less padding waste.
- **Neuron-specific notes:**
  - Prefill and decode never share a batch. The scheduler hides decode requests during prefill.
  - `max_prefills_per_batch` is fixed at 1 in 2.30; multiple waiting prefills run sequentially.

2: Use prefix caching

- **What it does:** Reuses the KV cache of shared prompt prefixes across requests, reducing time-to-first-token and increasing throughput for workloads with prefix overlap (system prompts, RAG contexts, few-shot demonstrations).
- **When to use:** Workloads where many requests share a common prefix — chatbots with system prompts, retrieval-augmented generation, or few-shot inference.
- **How to configure:**

Prefix caching is enabled by default in vLLM. No additional flags are required. To explicitly disable it:

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --no-enable-prefix-caching
```

- **Trade-offs:** Uses additional KV cache memory to store prefix blocks. Workloads with little or no prefix overlap will see negligible benefit.
- **Neuron-specific notes:**
  - Prefix caching leverages the segmented attention infrastructure (`kv_segment_size_buckets`) in the Neuron model runner.

3: Configure speculative decoding (Eagle3)

- **What it does:** Uses a small draft model to propose multiple tokens per step; the target model verifies them in parallel. Eagle3 is the supported method in 2.30.
- **When to use:** Decode-bound workloads (chat, agent loops). Does not help workloads bounded by prefill or extreme batch sizes.
- **How to configure:**

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --speculative-config \
        '{"method": "eagle3",
        "model": "<path-to-eagle3-draft>",
        "num_speculative_tokens": 4}'
```

- **Trade-offs:** Adds memory pressure (draft model on the same cores) and small per-step verification overhead. If draft acceptance is low for your workload, throughput can regress. Validate with a representative trace before production.
- **Neuron-specific notes:**
  - Mutually exclusive with async scheduling. Setting `--speculative-config` disables async scheduling automatically with a startup warning.
  - For supported model and draft combinations, see Reference: vLLM on Neuron feature and model compatibility matrix.
  - For a worked example, see Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron.

4: Configure quantization

- **What it does:** Two distinct features:
  - **FP8 KV cache** — Stores the KV cache in 8-bit float. Reduces KV memory.
  - **MXFP4 weights** — Stores model weights in mixed-precision FP4. Reduces model memory and improves compute throughput on Trn3.
- **When to use:**
  - **FP8 KV cache:** When KV memory is the bottleneck (lets you raise `--max-num-seqs` or `--max-model-len`).
  - **MXFP4:** For gpt-oss on Trn3 when you want maximum throughput.
- **How to configure:**

  FP8 KV cache (Llama and gpt-oss):

  ```
  vllm serve <model> --kv-cache-dtype fp8
  ```

  Requires an FP8-compatible KV checkpoint; the vLLM-Neuron examples include a `build_fp8_kv_checkpoint.py` helper.

  MXFP4 (gpt-oss on Trn3):

```
vllm serve openai/gpt-oss-120b \
    --tensor-parallel-size 64 \
    --additional-config '{"neuron_config": {"quantization": "mxfp4"}}'
```

- **Trade-offs:** Trades a small amount of accuracy for memory and throughput. Always validate with the How to debug accuracy issues with vLLM on Neuron after enabling quantization.
- **Neuron-specific notes:**
  - MXFP4 is Neuron-specific. Pass it under `neuron_config.quantization`, not the upstream `--quantization` flag.
  - The worker rejects `mxfp4` on Trn2. This fails at startup with a clear error.
  - INT8 weight quantization is not surfaced as a user feature in 2.30. Track its status in the **`/release-notes/index`**.
  - For per-model support, see Reference: vLLM on Neuron feature and model compatibility matrix.

5: Configure tensor, data, and expert parallelism

- **What it does:** Three parallelism dimensions:
  - **Tensor parallelism (TP)** — Shards each weight matrix across ranks. Required to fit large models.
  - **Data parallelism (DP)** — Runs independent model replicas. Increases concurrent throughput.
  - **Expert parallelism (EP)** — Distributes MoE experts across ranks. For MoE models at high core counts.
- **When to use:**
  - **TP:** Always. Pick the smallest size that fits the model and meets target latency.
  - **DP:** When you have spare cores beyond what TP requires.
  - **EP:** For MoE models when intermediate sharding becomes too small under high TP.
- **How to configure:**

```
# TP only, single replica
vllm serve meta-llama/Llama-3.3-70B-Instruct \
    --tensor-parallel-size 64

# TP + DP, multiple replicas
vllm serve openai/gpt-oss-20b \
    --tensor-parallel-size 8 \
    --data-parallel-size 8

# TP + EP, full expert parallelism for MoE
vllm serve openai/gpt-oss-20b \
    --tensor-parallel-size 8 \
    --enable-expert-parallel

# TP + EP with explicit ep_degree
vllm serve openai/gpt-oss-20b \
    --tensor-parallel-size 8 \
    --enable-expert-parallel \
    --additional-config '{"neuron_config": {"ep_degree": 2}}'
```

- **Trade-offs:** Each dimension adds collective traffic. EP reduces per-rank memory at the cost of routing all tokens through the EP collective. DP increases throughput linearly with replicas, at the cost of cores per replica. Match the parallelism to your bottleneck.
- **Neuron-specific notes:**
  - With `--enable-expert-parallel` and no explicit `ep_degree`, the worker defaults `ep_degree` to `--tensor-parallel-size` (full EP).
  - Setting `ep_degree > 1` without `--enable-expert-parallel` fails at startup.
  - Combining DP with EP requires `ep_degree = TP × DP`. Other combinations are rejected.

6: Configure disaggregated inference

- **What it does:** Splits prefill and decode across different processes (often different instances). Prefill workers stream KV state to decode workers via a KV connector. Patterns are called 1P1D (one prefill, one decode) and xPyD (multiple of each).
- **When to use:** Latency-sensitive deployments where prefill bursts degrade decode SLOs.
- **How to configure:** Use standard vLLM `--kv-transfer-config` flags. The Neuron plugin handles HBM-to-RDMA mapping internally.

```
# Prefill worker
vllm serve meta-llama/Llama-3.3-70B-Instruct \
    --tensor-parallel-size 32 \
    --kv-transfer-config \
        '{"kv_connector": "<connector>",
        "kv_role": "kv_producer"}'

# Decode worker
vllm serve meta-llama/Llama-3.3-70B-Instruct \
    --tensor-parallel-size 32 \
    --kv-transfer-config \
        '{"kv_connector": "<connector>",
        "kv_role": "kv_consumer"}'
```

  End-to-end setup is covered in the tutorials:

  - Tutorial: Configure disaggregated inference with 1P1D and xPyD on vLLM on Neuron — 1P1D and xPyD patterns.
- **Trade-offs:** Adds operational complexity (multiple processes, network transfer, orchestration) in exchange for stable decode latency.
- **Neuron-specific notes:**
  - Component-level data parallelism (per-layer DP for attention, MLP, embedding, or LM head) requires disaggregated inference. The worker rejects component DP without a `--kv-transfer-config`.

7: Use async scheduling and async execution

- **What it does:** Overlaps Python-level scheduling with Neuron device execution. Submits the next batch to the device while the previous batch's output is materialized on the CPU.
- **When to use:** On by default when on-device sampling is configured. Disable with `--no-async-scheduling` to debug scheduling issues.
- **How to configure:** No flag to enable. Two requirements must hold:
  - On-device sampling must be configured. Pass at least `{"all_greedy": true}` (or richer sampling config) under `neuron_config.on_device_sampling_config`.
  - Speculative decoding must be off. Setting `--speculative-config` disables async scheduling automatically.

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --additional-config \
        '{"neuron_config":
        {"on_device_sampling_config":
        {"all_greedy": true}}}'

# To force the synchronous path (for debugging):
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --no-async-scheduling
```

- **Trade-offs:** Improves throughput by hiding Python overhead. Cost is harder-to-debug timing because token output is materialized one step late.

- **Neuron-specific notes:**
  - vLLM upstream does not require on-device sampling; Neuron does.
  - The async scheduler falls back to synchronous execution on batch composition changes (request finishing, prefill-to-decode transition, shape changes).
  - Single in-flight NEFF execution (`NEURON_RT_ASYNC_EXEC_MAX_INFLIGHT_REQUESTS=1`).

8: Use structured outputs and tool calling

- **What it does:** Constrains generation to a JSON schema, regex pattern, choice enum, or BNF grammar. Tool calling reuses the same machinery: vLLM compiles tool schemas into a grammar bitmask that gates token sampling on the device.
- **When to use:** Workloads needing guaranteed-valid outputs (function calling, structured extraction, agent tool selection).
- **How to configure:** API is unchanged from upstream vLLM:

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")
client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[{"role": "user", "content": "Pick a tool"}],
    tools=[...],
    tool_choice="required",
)
```

- **Trade-offs:** Guarantees schema validity at the cost of a one-time recompile (~8 seconds) on the first constrained request, plus a small per-step bitmask transfer. Warm requests pay no extra cost.
- **Neuron-specific notes:**
  - On-device sampling required (same requirement as async scheduling). With CPU sampling, a constrained-output request raises `NotImplementedError` at runtime.
  - `--guided-decoding-backend` defaults to `xgrammar` on Neuron.

9: Customize warmup and the compilation cache

- **What it does:** On first launch, compiles a NEFF for every bucket combination. Compiled NEFFs are cached on disk and reused on later launches.
- **When to use:** On by default. Tune for production deployments where startup time or cache portability matters.
- **How to configure:** Set environment variables:
  - `VLLM_CACHE_ROOT` (default: `~/.cache/vllm`) — Point the cache at a larger or shared volume.
  - `VLLM_NEURON_REMOTE_CACHE` (default: unset) — NFS or FSx path so multiple instances share a compile cache.
  - `VLLM_NEURON_DISABLE_WARMUP_COMPILE=true` — Fail fast on a cache miss instead of compiling at startup.

```
export VLLM_CACHE_ROOT=/mnt/fast-nvme/vllm-cache
export VLLM_NEURON_REMOTE_CACHE=/mnt/fsx/shared-neuron-cache
export VLLM_NEURON_DISABLE_WARMUP_COMPILE=true

vllm serve meta-llama/Llama-3.3-70B-Instruct --tensor-parallel-size 64
```

- **Trade-offs:** A persistent shared cache cuts cold-start time at the cost of a network round-trip on lookups. Disabling warmup compilation makes startup deterministic in production but turns any missing bucket into a hard failure.

Confirm your work

For each feature you enable, validate behavior with a small targeted test before rolling the configuration into production:

- **Continuous batching tuning:** Send a representative concurrent load and confirm decode-batch utilization with vLLM's metrics.
- **Speculative decoding:** Compare decode latency with and without `--speculative-config`. Confirm the accepted-token ratio is above the break-even point for your workload.
- **Quantization:** Run the How to debug accuracy issues with vLLM on Neuron flow against your reference outputs.
- **Parallelism:** Confirm core utilization with `neuron-top` and that the model loads in expected memory.
- **Disaggregated inference:** Send prefill bursts and confirm decode latency stays within target.
- **Structured outputs:** Send a constrained-output request, observe the one-time recompile, and confirm subsequent requests are fast.

Common issues

A feature is silently disabled

- **Possible solution:** Some features have hard prerequisites (on-device sampling, supported model architecture, instance type). Check Reference: vLLM on Neuron feature and model compatibility matrix and the engine logs at startup. The startup banner lists which features are active.

`--enable-prefix-caching` does not appear to do anything

- **Possible solution:** Prefix caching is not supported in vLLM on Neuron 2.30. The flag is accepted but has no effect on the Neuron path. See the prefix caching section above and the **`/release-notes/index`** for status.

Performance regressed after enabling a feature

- **Possible solution:** Profile with and without the feature using a representative traffic mix. Speculative decoding can regress throughput if acceptance is low; quantization can regress accuracy enough to invalidate results.

Server fails with "ep_degree requires –enable-expert-parallel"

- **Possible solution:** Set both `--enable-expert-parallel` and `ep_degree` together, or remove `ep_degree` from `neuron_config` entirely.

Server fails with "quantization='mxfp4' is not supported on TRN2"

- **Possible solution:** Use BF16 on Trn2, or use Trn3 for MXFP4. See the quantization section.

Related information

- Reference: vLLM on Neuron feature and model compatibility matrix — Lookup table for feature-by-model support.
- How to set up vLLM on Neuron — Install and configure vLLM on Neuron.
- How to debug accuracy issues with vLLM on Neuron — Diagnose accuracy issues if feature changes affect outputs.
- How to deploy gpt-oss with vLLM on Neuron — End-to-end gpt-oss deployment recipe.
- Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron — Worked Eagle3 deployment.
- Tutorial: Configure disaggregated inference with 1P1D and xPyD on vLLM on Neuron — Worked disaggregated inference deployment.

Reference: vLLM on Neuron feature and model compatibility matrix

This topic provides a reference for which features are supported with which models when running vLLM on Neuron. Use this page to look up whether a given combination of model and feature is supported in the current Neuron 2.30 release.

Note

Last verified against the Neuron 2.30 release. Compatibility data goes stale fast. Check the **`/release-notes/index`** for the current information before you commit to a deployment.

Applies to

- vLLM on Neuron 2.30 deployments on AWS Trainium (Trn2 and Trn3 instance families).
- Model selection and feature planning for production rollouts.

Note

vLLM on Neuron 2.30 is supported on Trainium only. Inferentia (Inf2) is not supported in this release.

Overview

Feature support on vLLM on Neuron depends on three things: the upstream vLLM release, the vLLM-Neuron plugin version, and the specific model architecture. This page consolidates the supported combinations so you can confirm compatibility before you commit to a deployment path.

For how-to guidance on each feature, see How to use vLLM on Neuron features in your deployment. For per-model deployment guidance, see How to deploy gpt-oss with vLLM on Neuron.

Usage

Use this reference when:

- Selecting a model and checking which features apply to it.
- Investigating why a feature is not active in your deployment.
- Planning a migration or upgrade and validating feature parity.

Feature support by model

Cells use the following conventions:

- **Yes** — Supported and demonstrated in the published examples or recipes for Neuron 2.30.
- **No** — Not supported in Neuron 2.30. The combination may run, may fail at startup, or may be silently disabled. Do not rely on it.
- A short note (for example, FP8 KV) — supported with the qualifier indicated. See the matching footnote for details.

Each feature column header links to the matching section in the How to use vLLM on Neuron features in your deployment.

Model	Tested instances [1]	Continuous batching	Prefix caching [2]	Speculative decoding (Eagle3)	Quantization	Multimodal	Disaggregated inference	Tool calling / structured outputs [8]
Llama 3.x dense (3.1, 3.2, 3.3) [3]	Trn2	Yes	Yes	Yes [4]	FP8 KV [6]	No	Yes	Yes
gpt-oss 20B	Trn2, Trn3	Yes	Yes	Yes [5]	FP8 KV [6]; MXFP4 (Trn3) [7]	No	Yes	Yes

gpt-oss 120B	Trn2, Trn3	Yes	Yes	Yes [5]	FP8 KV [6]; MXFP4 (Trn3) [7]	No	Yes	Yes

Notes on conditional support

1 "Tested instances" lists the Trainium instance families exercised by the published examples and recipes for Neuron 2.30. Other Trainium configurations may work but are not validated for this release.

2 Prefix caching is enabled by default in vLLM on Neuron 2.30. It reuses KV cache blocks for shared prompt prefixes across requests.

3 The Llama 3.x row covers `meta-llama/Llama-3.1-8B-Instruct`, `meta-llama/Llama-3.2-1B-Instruct`, and `meta-llama/Llama-3.3-70B-Instruct`. Other Llama 3.x checkpoints that use the same architecture (`LlamaForCausalLM`) are expected to work but are not individually validated.

4 Eagle3 speculative decoding is demonstrated for Llama 3.1 8B Instruct in the published examples. Eagle3 requires a compatible draft checkpoint. See Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron.

5(1, 2) Eagle3 speculative decoding is demonstrated for gpt-oss 120B in the published examples and is expected to work for gpt-oss 20B with a compatible draft checkpoint. See Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron.

6(1, 2, 3) FP8 quantization is supported for the **KV cache only** (`--kv-cache-dtype fp8`). Use the FP8 KV checkpoint builder included in the vLLM-Neuron examples to prepare a compatible checkpoint. INT8 weight quantization is not surfaced in the Neuron 2.30 examples and is not covered by this matrix; check the **`/release-notes/index`** for current status.

7(1, 2) MXFP4 quantization is supported for gpt-oss on Trn3 only. On Trn2, the gpt-oss path falls back to BF16; explicitly requesting `--quantization mxfp4` on Trn2 will fail at startup.

8 Tool calling and structured outputs (JSON schema, regex, choice, BNF grammar) are vLLM features applied at the sampler layer. Behavior is independent of the model architecture in Neuron 2.30. See How to use vLLM on Neuron features in your deployment for configuration.

Related reference

- How to use vLLM on Neuron features in your deployment — How to use each feature in your deployment.
- How to set up vLLM on Neuron — Install and configure vLLM on Neuron.
- How to deploy gpt-oss with vLLM on Neuron — End-to-end deployment recipe for gpt-oss.

Learn more

- **`/release-notes/index`** — Authoritative source for the latest supported features and models.
- Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron — Deploy Eagle3 speculative decoding.
- Tutorial: Configure disaggregated inference with 1P1D and xPyD on vLLM on Neuron — Configure disaggregated inference (1P1D / xPyD).

How to onboard a model to vLLM on Neuron

Task overview

This topic discusses how to onboard a new model architecture so it can be served through vLLM on Neuron. You will implement the model using the vLLM-Neuron plugin's building blocks, register it with vLLM's model registry, compile, validate accuracy, and benchmark performance.

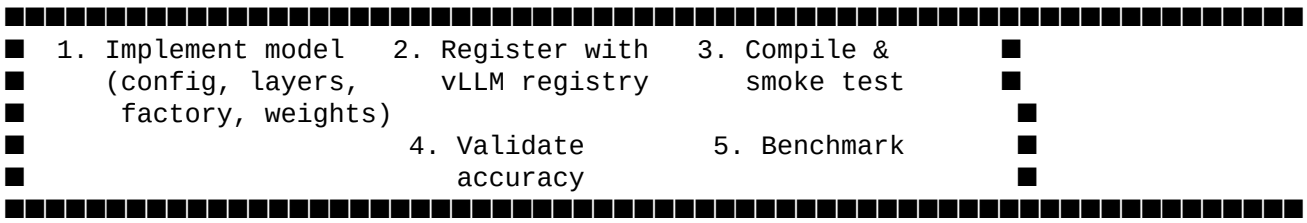
> **Note**
>
> The vLLM-Neuron plugin implements models directly using `libtorch-neuron-lite`, NKI (Neuron Kernel Interface) kernels, and its own parallel infrastructure. The plugin provides its own functional operators (`vllm_neuron.functional`), neural network modules (`vllm_neuron.nn`), weight loading utilities, and parallel state management built on top of `torch.distributed` and vLLM's distributed primitives.

> **Note**
>
> For models deployed through NxD Inference directly (without vLLM), see the legacy **`NxDI model onboarding guide`**.

Prerequisites

- **vLLM on Neuron environment:** A working vLLM on Neuron setup on a Trainium or Inferentia instance. See How to set up vLLM on Neuron.
- **A model you want to onboard:** You may be porting from an existing implementation (e.g., Hugging Face Transformers), writing from scratch, or adapting from another framework. Any approach works as long as the result satisfies the runner interface described below.
- **Familiarity with the plugin codebase:** The vLLM-Neuron plugin source contains annotated reference implementations (Llama, GPT-OSS) that demonstrate common patterns. These are useful as examples, not mandatory templates.

End-to-end onboarding flow



The Llama implementation (`vllm_neuron/model/llama3/`) is heavily annotated with `# <-- MODEL-SPECIFIC` and `# >>> PARALLELISM <<<` comments to distinguish model-specific code from reusable parallelism infrastructure. Use it as your primary porting template.

Instructions

1: Implement the model

Create a model directory under `vllm_neuron/model/` with the following structure:

```
vllm_neuron/model/your_model/
├── __init__.py
├── config.py          # Model-specific dataclass config
├── factory.py         # Factory class for vLLM ModelRegistry
└── model.py           # Full model implementation
```

1a: Define the model config (`config.py`)

Create a dataclass that holds the model's architecture parameters. Implement a `from_configs` classmethod that parses a HuggingFace `PretrainedConfig` into your config.

```python
from dataclasses import dataclass
import torch
from transformers import PretrainedConfig
from vllm_neuron.model.neuron_config import NeuronConfig

@dataclass
class YourModelConfig:
    # Architecture parameters (model-specific)
    hidden_size: int = 4096
    num_attention_heads: int = 32
    num_key_value_heads: int = 8
    num_hidden_layers: int = 32
    intermediate_size: int = 14336
    head_dim: int = 128
    vocab_size: int = 128256
    max_position_embeddings: int = 131072
    rms_norm_eps: float = 1e-5
    rope_theta: float = 500000.0
    rope_scaling: dict | None = None
    tie_word_embeddings: bool = False
    torch_dtype: torch.dtype = torch.bfloat16

    # Framework config
    neuron_config: NeuronConfig | None = None

    @classmethod
    def from_configs(
        cls,
        hf_config: PretrainedConfig,
        neuron_config: NeuronConfig = None,
    ):
        config_dict = hf_config.to_dict()
        field_names = {
            f.name for f in cls.__dataclass_fields__.values()
        }
        filtered_dict = {
            k: v for k, v in config_dict.items()
            if k in field_names
        }
        if neuron_config is not None:
            filtered_dict["neuron_config"] = neuron_config
        return cls(**filtered_dict)
```

The `NeuronConfig` dataclass (provided by the plugin at `vllm_neuron.model.neuron_config`) carries Neuron-specific settings: parallelism degrees (`attention_dp_size`, `mlp_dp_size`, `embedding_dp_size`, `lm_head_dp_size`, `ep_degree`), on-device sampling config, bucket configurations, and quantization settings.

1b: Implement model components (`model.py`)

Your model implementation can use any valid PyTorch code that the Neuron compiler can trace. The plugin provides optimized building blocks (functional operators backed by NKI kernels, parallel modules, weight loading utilities) that you can use if they fit your architecture. These are conveniences, not requirements — you are free to write your own operators, use different parallelism strategies, or structure your model however you like, as long as the result satisfies the runner interface contract described below.

Available building blocks:

The following utilities are provided by the plugin and used by the existing model implementations. They handle common patterns efficiently on Neuron:

Component	Available utilities

Attention	`vllm_neuron.functional.qkv_proj`, `flash_attention`, `segmented_attention`, `attention_decode`, `o_proj`, `gen_attention_decode_mask` — NKI-backed operators with PyTorch CPU fallbacks for tracing
MLP	`vllm_neuron.functional.mlp` — fused gate/up/down projection with NKI kernel
Embedding	`vllm_neuron.nn.VocabDimShardedEmbedding` — vocab-dimension-sharded embedding with built-in scatter for SP
LM Head	`vllm_neuron.nn.ColumnParallelLinear` — column-parallel linear with optional output gathering
Normalization	Standard `nn.Module` implementations (RMSNorm, LayerNorm) — no special Neuron module needed
Weight loading	`vllm_neuron.utils.weight_loader.sharding_weight_loader`, `fused_qkv_weight_loader`, `with_rank_override` — closure-based loaders for checkpoint → parameter transforms
Collectives	`vllm.distributed.parallel_state.get_tp_group()` — provides `all_gather`, `reduce_scatter`, `all_reduce`; plugin also provides custom group coordinators in `vllm_neuron.parallel.neuron_parallel_state`
KV cache	`vllm_neuron.model.kv_cache.KVSpec`, `LayerSpec` — dataclasses for declaring cache requirements

Note

The functional operators in `vllm_neuron.functional` (imported as `NF`) dispatch to NKI kernels when running on Neuron hardware and fall back to equivalent PyTorch operations on CPU. This dual-path design allows `torch.compile` to trace the graph on CPU while the compiled NEFF uses optimized kernels at runtime. You can use these operators, write your own, or mix both.

Understanding the runner → model interface

The `NeuronModelRunner` orchestrates execution between vLLM's scheduler and your model. Understanding what it passes to `forward()` is essential for implementing a correct model.

Forward signature:

Your top-level model's `forward()` receives these keyword arguments (all are always present once torch.compile traces the graph — you cannot conditionally accept/reject kwargs between calls):

```
def forward(
    self,
    # [total_tokens] token IDs, padded to bucket size
    input_ids: torch.LongTensor,
    # [total_tokens] position IDs (int32)
    positions: torch.Tensor,
    # Per-layer attention metadata (see below)
    attn_metadata: dict,
    # [num_reqs] indices into token dim for LM head
    sampling_positions: torch.Tensor,
    # [num_reqs, 3] [top_k, top_p, temperature]
    sampling_params: torch.Tensor,
    # SpecDecodeMetadata for rejection sampling
    spec_decode_metadata=None,
    # [num_logit_rows, vocab_size] grammar mask
    logit_mask: torch.Tensor,
    # scalar TP rank
    rank: torch.Tensor,
    # [total_tokens, hidden_size] prompt-embeds
```

```
    inputs_embeds: torch.Tensor | None = None,
    # [total_tokens] bool (prompt-embeds path)
    is_token_ids: torch.Tensor | None = None,
) -> torch.Tensor | tuple:
```

Attention metadata structure:

`attn_metadata` is a **dict keyed by layer name** (`"layers.0.self_attn"`, `"layers.1.self_attn"`, etc.). Each attention layer indexes into it by its own name. The per-layer dict contains:

Key	Description
`block_table_tensor`	`[padded_num_reqs, max_blocks_per_seq]` int32 — maps (request, block_index) → physical block ID
`slot_mapping`	`[total_tokens]` int64 — maps each token position → KV cache slot. Padding tokens have value `-1`.
`max_query_len`	int — maximum tokens scheduled per request in this batch
`block_size`	int — number of positions per KV cache block
`max_blocks_per_seq`	int — max blocks allocated per sequence (equals `block_table_tensor.shape[1]`)
`decode_token_threshold`	int — if `max_query_len <= decode_token_threshold`, use the decode path
`cached_seq_len`	`[[int]]` tensor — number of previously cached tokens (for segmented prefill)
`kv_segment_size`	int — 0 = full prefill; >0 = segmented prefill chunk size

Prefill vs. decode dispatch:

Each attention layer dispatches between its prefill and decode paths using:

```
layer_name = f"layers.{self.layer_idx}.self_attn"
layer_meta = attn_metadata[layer_name]
max_query_len = layer_meta["max_query_len"]
decode_token_threshold = (
    layer_meta["decode_token_threshold"]
)

if max_query_len <= decode_token_threshold:
    return self.forward_decode(...)
else:
    return self.forward_prefill(...)
```

Return values:

What `forward()` returns depends on on-device sampling (ODS) configuration:

Mode	Return value
CPU sampling (no ODS)	`logits` — `[num_reqs, vocab_size]` float tensor
CPU sampling + Eagle3	`(logits, aux_hidden_states)`
On-device sampling	`(sampled_token_ids, gathered_logits)`
On-device sampling + Eagle3	`(sampled_token_ids, aux_hidden_states, gathered_logits)`

Things to be aware of:

- **Padding is real.** `input_ids` is padded to compiled bucket sizes. The model processes all tokens including padding. Padding tokens have `slot_mapping = -1` — if your model writes to the KV cache, skip slots with value `-1`.

- ``**sampling_positions**`` tells the model which token positions to produce logits for. The existing models use `torch.index_select` on the final hidden states to select only these positions before the LM head projection, avoiding unnecessary computation.
- **The model owns KV cache writes.** The runner allocates cache blocks and binds them via `bind_kv_cache()`, but it's up to the model's attention implementation to actually write K/V into paged blocks using `slot_mapping`, `block_size`, and computed block/position indices.
- **Forward signature is fixed at compile time.** `torch.compile` traces a static graph. All kwargs that appear in any call must always be present — you cannot conditionally accept different kwargs between calls.
- **Conditional paths create separate compiled graphs.** Python-level branching (e.g., prefill vs. decode dispatch based on `attn_metadata` values) is normal — each branch is traced into its own NEFF during warmup. After warmup, `torch.compiler.set_stance("fail_on_recompile")` prevents new graph variants. Data-dependent control flow (branching on tensor *values*) will break compilation because tensor values aren't known at trace time.

Attention class pattern (example):

The existing models implement attention with these patterns. Your implementation can differ, but these illustrate common choices:

- **TP head sharding:** Q/K/V heads divided across TP ranks. GQA replication when KV heads < TP size.
- **Two execution paths:** Prefill (flash attention with SP all-gather/reduce-scatter) and decode (fused megakernel via `NF.attention_decode`). These are separate code paths dispatched based on `attn_metadata`.
- **KV cache writes:** The model writes new K/V into the paged cache using `slot_mapping` from `attn_metadata`. This is the model's responsibility — the runner does not write to the cache.
- **Weight loaders:** Closures attached to parameters that handle checkpoint → parameter transforms (sharding, fusion).

```
import vllm_neuron.functional as NF
from vllm.distributed.parallel_state import get_tp_group
from vllm_neuron.utils.weight_loader import (
    fused_qkv_weight_loader,
    sharding_weight_loader,
    set_weight_loader,
)

class YourModelAttention(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.tp_group = get_tp_group()
        self.world_size = self.tp_group.world_size
        # Shard Q/K/V heads across TP ranks
        self.num_attention_heads_per_rank = (
            config.num_attention_heads // self.world_size
        )
        self.num_key_value_heads_per_rank = max(
            1, config.num_key_value_heads // self.world_size
        )
        # ... define qkv_proj_weight, o_proj_weight params
        # Attach weight loaders for TP sharding
        set_weight_loader(
            self.qkv_proj_weight,
            fused_qkv_weight_loader(...),
        )

    def forward(
        self, hidden_states, positions,
        position_embeddings, attn_metadata,
```

```
    ):
        # Dispatch to prefill or decode based on attn_metadata
        ...
```

MLP class pattern:

```
class YourModelMLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.tp_group = get_tp_group()
        self.intermediate_size_per_rank = (
            config.intermediate_size
            // self.tp_group.world_size
        )
        # Define gate_proj_weight, up_proj_weight, down_proj_weight
        # Attach sharding weight loaders
        ...

    def forward(self, hidden_states, is_prefill):
        if is_prefill and self.tp_group.world_size > 1:
            hidden_states = self.tp_group.all_gather(
                hidden_states, dim=0
            )
        output = NF.mlp(
            hidden_states,
            self.gate_proj_weight,
            self.up_proj_weight,
            self.down_proj_weight,
        )
        if is_prefill:
            output = self.tp_group.reduce_scatter(
                output, dim=0
            )
        else:
            self.tp_group.all_reduce(output)
        return output
```

Top-level model class (``YourModelForCausalLM``):

This class implements the interface methods that the vLLM-Neuron runtime calls. These are the hard requirements — the runner expects them to exist:

```
from vllm_neuron.model.kv_cache import KVSpec, LayerSpec

class YourModelForCausalLM(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.model = YourModelBackbone(config)
        self.lm_head = neuron_nn.ColumnParallelLinear(...)
        ...

    def forward(
        self, input_ids, positions, attn_metadata,
        sampling_positions, sampling_params, ...
    ):
        """Run the full model: embed → layers → norm → LM head."""
        ...

    def get_kv_spec(self) -> KVSpec:
        """Declare the KV cache shape for each layer."""
        layers = []
        for i, layer in enumerate(self.model.layers):
```

```
            attn = layer.self_attn
            layers.append(LayerSpec(
                name=f"layers.{i}.self_attn",
                num_kv_heads=attn.num_key_value_heads_per_rank,
                head_size=attn.head_dim,
                dtype=attn.dtype,
            ))
        return KVSpec(layers=layers)

    def bind_kv_cache(
        self, kv_caches: dict[str, list[torch.Tensor]]
    ):
        """Bind allocated KV cache tensors to attn layers."""
        for i, layer in enumerate(self.model.layers):
            name = f"layers.{i}.self_attn"
            layer.self_attn.k_cache = kv_caches[name][0]
            layer.self_attn.v_cache = kv_caches[name][1]

    def load_weights(
        self, checkpoint_path, device, cache_dir
    ):
        """Load weights from safetensors with TP sharding."""
        ...

    @classmethod
    def from_configs(cls, hf_config, neuron_config):
        config = YourModelConfig.from_configs(
            hf_config, neuron_config
        )
        return cls(config)
```

KV cache lifecycle:

The KV cache is managed through a three-step contract between your model and the runner:

1. **Declaration (``get_kv_spec``)** — At startup, the runner calls your model's `get_kv_spec()` to learn the cache requirements for each layer: number of KV heads (per rank), head size, dtype, and optionally sliding window size. The runner uses this to compute how much memory to allocate and how to organize cache groups.
2. **Allocation and binding (``bind_kv_cache``)** — The runner allocates paged KV cache tensors on device and calls `bind_kv_cache(kv_caches)` on your model. `kv_caches` is a dict mapping layer names (e.g., `"layers.0.self_attn"`) to `[k_cache, v_cache]` pairs. Your model stores references to these tensors (typically on the attention layer). Each cache tensor has shape `[num_blocks, num_kv_heads, block_size, head_size]` (HND paged layout).
3. **Writing at runtime** — During forward passes, the model writes new K/V values into the cache using `slot_mapping` from `attn_metadata`. The slot mapping converts token positions into (block_index, position_within_block) pairs. Padding tokens have `slot_mapping = -1` and should not be written.

The layer names used in `get_kv_spec` must match the keys that the model reads from `attn_metadata` — the runner uses the same names for both. By convention, existing models use `f"layers.{i}.self_attn"`.

1c: Define the factory (`factory.py`)

The factory class satisfies vLLM's ModelRegistry interface and selects the correct implementation variant (e.g., BF16 vs. quantized).

```
import torch.nn as nn
from transformers import PretrainedConfig
from vllm_neuron.model.neuron_config import NeuronConfig

class YourModelForCausalLM(nn.Module):
    """Factory that validates config and selects implementation."""
```

```
    def __init__(
        self,
        hf_config: PretrainedConfig,
        neuron_config: NeuronConfig | None,
    ) -> None:
        super().__init__()
        self._model = self._select_implementation(
            hf_config, neuron_config
        )

    def forward(self, *args, **kwargs):
        return self._model(*args, **kwargs)

    @classmethod
    def from_configs(cls, hf_config, neuron_config):
        return cls._select_implementation(
            hf_config, neuron_config
        )

    @classmethod
    def _select_implementation(
        cls, hf_config, neuron_config
    ):
        cls._validate_config(hf_config, neuron_config)
        from .model import YourModelForCausalLM as Model
        return Model.from_configs(
            hf_config, neuron_config
        )

    @classmethod
    def _validate_config(cls, hf_config, neuron_config):
        # Add validation rules
        pass
```

1d: Implement weight loading

Weight loading connects checkpoint tensors (stored in safetensors format) to your model's parameters, applying any necessary transforms along the way (sharding for TP, fusing separate weights, transposing). The plugin provides utilities for this, but you can also implement `load_weights` however you prefer.

How the weight loader system works:

The flow has three parts: a **mapping** (checkpoint key → parameter name), **weight loaders** (transform closures attached to parameters), and a **checkpoint reader** that ties them together.

1. **Mappings** — A dict that maps your model's parameter names (as they appear in `state_dict()`) to one or more checkpoint keys. If a parameter comes from multiple checkpoint tensors (e.g., fused QKV from separate Q, K, V), map it to a list.
2. **Weight loaders** — A `SafetensorsWeightLoader` attached to each parameter via `set_weight_loader()`. It contains a `transform` function: `(slices: list[PySafeSlice], rank: int) -> torch.Tensor`. The `slices` are lazy safetensors references (one per checkpoint key from the mapping), and `rank` is the current TP rank. The transform extracts the rank's shard.
3. **SafetensorsCheckpoint** — Reads safetensors files without loading everything into memory. `load_sharded_pipelined()` iterates over your mappings, finds each checkpoint key, passes slices to the parameter's weight loader, and collects the results into a state dict.

Example: simple sharding weight loader

`sharding_weight_loader` creates a transform that slices a single checkpoint tensor along a dimension:

```
from vllm_neuron.utils.weight_loader import (
    sharding_weight_loader, set_weight_loader,
```

```
)

# Shard gate_proj along dim 1 (output features) across TP
loader = sharding_weight_loader(
    shard_dim=1,
    shard_size=intermediate_size // tp,
    num_shards=tp,
    is_storage_transposed=True,
)
set_weight_loader(self.gate_proj_weight, loader)
```

Example: fused QKV weight loader

`fused_qkv_weight_loader` handles the case where Q, K, V are stored as separate checkpoint tensors but your model uses a single fused parameter:

```
from vllm_neuron.utils.weight_loader import (
    fused_qkv_weight_loader, set_weight_loader,
)

loader = fused_qkv_weight_loader(
    q_size=num_q_heads_per_rank * head_dim,
    kv_size=num_kv_heads_per_rank * head_dim,
    shard_dim=1,
    num_shards=tp_size,
    is_storage_transposed=True,
    # >1 when KV heads < TP size (GQA)
    num_kv_replicas=num_kv_replicas,
)
set_weight_loader(self.qkv_proj_weight, loader)
```

Example: rank override for component DP

When a weight should be sharded across a larger group (e.g., TP × attention_dp), use `with_rank_override` to change the effective rank the loader uses:

```
from vllm_neuron.utils.weight_loader import (
    with_rank_override,
)

effective_rank = (
    attention_dp_rank + tp_rank * attention_dp_size
)
loader = with_rank_override(
    sharding_weight_loader(...), rank=effective_rank
)
set_weight_loader(self.o_proj_weight, loader)
```

Writing a custom weight loader:

If the provided utilities don't fit your needs, write a custom `SafetensorsWeightLoader`:

```
from vllm_neuron.utils.weight_loader import (
    SafetensorsWeightLoader, set_weight_loader,
)

def my_custom_loader(slices, rank):
    # slices: list of PySafeSlice objects
    #   (lazy references to checkpoint tensors)
    # rank: current TP rank
    # Return: the final tensor for this param on this rank
    full_tensor = slices[0][:]  # Load the full tensor
    # ... do whatever transform you need ...
    shard_start = rank * shard_size
```

```
    shard_end = (rank + 1) * shard_size
    return full_tensor[shard_start:shard_end]

set_weight_loader(
    self.some_param,
    SafetensorsWeightLoader(transform=my_custom_loader),
)
```

Putting it all together in ``load_weights``:

```
from vllm_neuron.utils.checkpoints import (
    SafetensorsCheckpoint,
)

def load_weights(
    self, checkpoint_path, device, cache_dir
):
    # Define checkpoint key -> parameter name mappings
    mappings = {}
    for layer_id in range(len(self.model.layers)):
        prefix = f"model.layers.{layer_id}"
        # Fused QKV: 3 checkpoint keys -> 1 parameter
        qkv_key = f"{prefix}.self_attn.qkv_proj_weight"
        mappings[qkv_key] = [
            f"{prefix}.self_attn.q_proj.weight",
            f"{prefix}.self_attn.k_proj.weight",
            f"{prefix}.self_attn.v_proj.weight",
        ]
        # 1:1 mappings
        attn = f"{prefix}.self_attn"
        mlp = f"{prefix}.mlp"
        mappings[f"{attn}.o_proj_weight"] = (
            f"{attn}.o_proj.weight"
        )
        mappings[f"{mlp}.gate_proj_weight"] = (
            f"{mlp}.gate_proj.weight"
        )
        mappings[f"{mlp}.up_proj_weight"] = (
            f"{mlp}.up_proj.weight"
        )
        mappings[f"{mlp}.down_proj_weight"] = (
            f"{mlp}.down_proj.weight"
        )
        # Norms: no sharding (loaded as-is)
        mappings[f"{prefix}.input_layernorm.weight"] = (
            f"{prefix}.input_layernorm.weight"
        )
        mappings[
            f"{prefix}.post_attention_layernorm.weight"
        ] = f"{prefix}.post_attention_layernorm.weight"

    # Open checkpoint and load with per-param transforms
    checkpoint = SafetensorsCheckpoint(
        checkpoint_path, cache_dir
    )
    rank_sharded = checkpoint.load_sharded_pipelined(
        self.rank, self.world_size, self, mappings, device
    ).state_dict

    self.load_state_dict(
```

```
            rank_sharded, strict=False, assign=True
        )
```

Parameters without a weight loader attached are loaded from the checkpoint as-is (no transform). This is appropriate for unsharded weights like norm scales.

2: Register the model with vLLM

Add your model to the plugin's registry so vLLM discovers it at worker initialization.

Edit `vllm_neuron/model/registry.py`:

```
from .your_model import YourModelForCausalLM

def get_models() -> list[tuple[str, type]]:
    models = [
        # ... existing models ...
        ("YourModelForCausalLM", YourModelForCausalLM),
    ]
    return models
```

The string key (`"YourModelForCausalLM"`) must match the `architectures` field in the model's `config.json` on HuggingFace. When the vLLM Neuron worker starts, it registers all models from this list with vLLM's `ModelRegistry`.

3: Compile and run a smoke test

Run a first inference to trigger Neuron compilation and verify the model loads.

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="/path/to/model",
    max_num_seqs=4,
    max_model_len=2048,
    tensor_parallel_size=8,
)

output = llm.generate(
    ["Hello, my name is"],
    SamplingParams(max_tokens=32, temperature=0.0),
)
print(output[0].outputs[0].text)
```

Understanding warmup and compilation:

Before the model can serve real requests, the runner executes a **warmup phase**. During warmup, it calls your model's `forward()` with synthetic inputs for every bucket size — each prefill sequence length bucket and each decode batch size bucket. Each call triggers `torch.compile` to trace the graph for that input shape and compile it into a NEFF (Neuron Executable File Format). This means:

- A separate NEFF is compiled for each unique combination of (prefill bucket size, kv_segment_size) and each (decode batch size).
- Conditional paths (prefill vs. decode) are traced separately per bucket — each bucket's warmup call exercises one branch.
- After warmup completes, `fail_on_recompile` is set — any input shape not covered by the warmup buckets will fail rather than silently recompiling.
- Compiled NEFFs are cached on disk. Subsequent server starts with the same model, bucket config, and TP degree will skip compilation and load from cache.

Bucket sizes are controlled via NeuronConfig (`num_batched_tokens_buckets` for prefill, `num_seqs_buckets` for decode). If not specified, defaults are computed from `max_model_len` and `max_num_seqs`. More buckets means more compilations during warmup but better padding efficiency at runtime.

Runtime padding to buckets:

At runtime, actual inputs are padded up to the nearest compiled bucket size before being passed to the model. For prefill, the token sequence is padded to the next `num_batched_tokens_buckets` entry. For decode, the batch is padded to the next `num_seqs_buckets` entry. This means:

- Your model always receives inputs at one of the pre-compiled bucket sizes — never the raw unpadded size.
- Padding tokens use `pad_token_id` for `input_ids` and have `slot_mapping = -1` so they don't write to the KV cache.
- `sampling_positions` only points at real (non-padding) token positions, so padding doesn't affect output correctness.
- Fewer buckets means more padding waste (the gap between actual size and the next bucket can be large). More buckets means less waste but longer warmup.

4: Validate accuracy

Verify that the model produces correct outputs. The most common approach is comparing against a reference (e.g., the same model running on GPU via Hugging Face Transformers), but you can use whatever validation method makes sense for your model — evaluation benchmarks, known-answer prompts, perplexity checks, etc.

If comparing against a reference:

1. Run the same prompt through vLLM on Neuron and the reference, both with greedy sampling (`temperature=0.0`).
2. Compare generated tokens. For large models, token-level divergence at later positions is expected — use logit-level comparison to confirm outputs are within tolerance.
3. If accuracy issues appear, use the systematic workflow in How to debug accuracy issues with vLLM on Neuron and the **`how-to-use-accuracy-debugger`** tool to isolate the cause.

Running on CPU for debugging:

You can run models on CPU by setting the environment variable `VLLM_NEURON_CPU_MODE=1`. No Neuron hardware or compilation is needed. This is useful for:

- Rapid iteration during development — no compile wait.
- Isolating whether an accuracy issue is in your model logic vs. the Neuron compilation. If outputs differ between CPU mode and on-device, the issue is likely in compilation or a kernel; if they match, the issue is in your model code.
- Running module tests locally without a Neuron instance (`VLLM_NEURON_CPU_MODE=1 pytest ...`).

By default in CPU mode, the functional operators (`vllm_neuron.functional`) use their PyTorch fallback implementations. You can also enable the NKI CPU simulator by setting `NKI_SIMULATOR=1`, which runs the actual NKI kernel logic on CPU — useful for validating kernel correctness without hardware.

Existing test patterns to follow:

The plugin includes two levels of accuracy tests for existing models that you can use as patterns for your own:

- **Module tests** (`test/vllm_neuron/model/<model>/modules/`) — Test individual components (attention, MLP, RoPE) in isolation against a HuggingFace reference. These load real checkpoint weights into a single module, run it with known inputs, and compare outputs. Useful for catching issues early without running the full model. Can run on CPU (`VLLM_NEURON_CPU_MODE=1`) or on device.
- **E2E logit tests** (`test/vllm_neuron/model/<model>/e2e/`) — Run the full model through vLLM (offline or online serving) and validate output logits against golden references across a grid of configurations (TP sizes, sequence lengths, batch sizes). These use `vllm_neuron.accuracy.logit_validation.multi_prompt_logit_validation` to compare multi-prompt logit outputs with tolerances.

When onboarding a new model, writing module tests for each component (attention, MLP, RoPE, etc.) and an e2e logit test gives you confidence that the implementation is correct and provides regression coverage going forward.

> **Note**
>
> The plugin provides a `tensor_capture` NeuronConfig option for capturing intermediate activations and a `tensor_replacement` option for injecting reference tensors layer-by-layer. These are valuable for isolating where drift is introduced.

5: Benchmark and tune performance

Once accuracy is validated, measure latency and throughput.

```
# Start the server
vllm serve /path/to/model \
    --tensor-parallel-size 8 \
    --max-num-seqs 32

# Benchmark (in a separate terminal)
python benchmarks/benchmark_serving.py \
    --model /path/to/model \
    --num-prompts 100
```

Key tuning parameters (passed via `--additional-config '{"neuron_config": {...}}'`):

- `tensor_parallel_size` — Number of NeuronCores for TP.
- `max_num_seqs` — Controls continuous batching concurrency.
- `max_model_len` — Affects compilation bucket sizes and memory usage.
- `attention_dp_size` — Shards Q/O across DP ranks during decode for better batch scaling.
- `mlp_dp_size` — Shards MLP intermediate across DP ranks.
- `on_device_sampling_config` — Enables on-device sampling to overlap sampling with next-token compute.
- `prefill_sequence_buckets` / `decode_batch_buckets` — Control compilation bucket sizes.

For more features you can enable after onboarding, see How to use vLLM on Neuron features in your deployment.

Confirm your work

To confirm the model is correctly onboarded:

1. **Compilation:** The model compiles without errors and loads from cache on subsequent runs.
2. **Accuracy:** Generated output is within tolerance of the reference implementation under greedy sampling.
3. **Performance:** Latency and throughput meet targets on the intended instance type.
4. **Serving:** The model serves requests correctly through both offline (`LLM.generate()`) and online (`vllm serve`) modes.

Common issues

Unsupported or non-compilable operations

- **Possible solution:** The plugin uses `torch.compile` to trace your model into a static graph that the Neuron compiler can lower to hardware. Everything in your `forward()` path must be compilable — no operations that break out of the graph. Common graph breaks include: calling `.item()` or `.tolist()` on a tensor (materializes data to Python), data-dependent control flow (`if tensor.sum() > 0`), in-place ops on views in some contexts, and Python-side print/logging that references tensor values. If you hit a graph break, the

error message from `torch._dynamo` will point at the offending line. Restructure to use only tensor operations that can be traced statically.

On conditional paths: Python-level branching based on metadata values (e.g., `if max_query_len <= decode_token_threshold`) does create separate compiled graphs — one per branch taken. This is expected and handled by the system: during warmup, each bucket size is traced and compiled into its own NEFF. Once warmup is complete, `torch.compiler.set_stance("fail_on_recompile")` is active, meaning no *new* graph variants are allowed at runtime. So conditional paths are fine as long as they are exercised during warmup (the runner handles this automatically for standard prefill/decode dispatch). What you cannot do is introduce conditionals that produce an unbounded or unexpected number of graph variants — each unique path is a separate compilation.

`KeyError` or missing key during weight loading

- **Possible solution:** Your checkpoint-to-parameter name mapping is incomplete. Check that your `mappings` dict in `load_weights` covers all parameters. For fused weights (e.g., separate Q/K/V → fused QKV), ensure the mapping is a list of checkpoint keys, not a single key. If a parameter name in your model doesn't match the checkpoint's naming convention, add the appropriate mapping entry. Parameters without a weight loader attached are loaded as-is — if the checkpoint key name differs from the parameter name, you'll get a `KeyError`.

Accuracy drift

- **Possible solution:** Start with the How to debug accuracy issues with vLLM on Neuron to isolate whether drift comes from attention, KV cache, weight loading, or sampling. Common culprits: incorrect RoPE variant (interleaved vs. split-half), wrong head dimension after sharding, dtype mismatches, and KV cache quantization scale errors. Use `tensor_capture` / `tensor_replacement` in NeuronConfig to inject reference tensors at specific module boundaries and narrow down which layer introduces the divergence.

Related information

- How to use vLLM on Neuron features in your deployment — Features you can enable once the model is onboarded.
- How to debug accuracy issues with vLLM on Neuron — Diagnose accuracy issues discovered during onboarding.
- Reference: vLLM on Neuron feature and model compatibility matrix — Check which features are compatible with your model.
- **`NxDI model onboarding guide`** — Legacy onboarding guide for NxDI-only deployments (without vLLM).
- vLLM-Neuron plugin source — Plugin source code with annotated reference implementations.

How to debug accuracy issues with vLLM on Neuron

Task overview

This topic discusses how to systematically diagnose and resolve accuracy issues when running models with vLLM on Neuron. It provides a workflow for isolating where accuracy drift is introduced — from checkpoint loading through generation — so you can fix the underlying cause rather than chasing symptoms.

Prerequisites

- **vLLM on Neuron deployment:** A running or reproducible deployment where you observe the accuracy issue. See How to set up vLLM on Neuron.
- **Reference output:** Known-good outputs from a reference implementation (CPU, GPU, or a prior Neuron release) that you can compare against.

- **Familiarity with the model:** Understanding of the model's architecture, attention variant, and tokenizer.
- **Python environment:** Access to HuggingFace Transformers and the model checkpoint for generating reference outputs.

Validation levels

The accuracy debugging framework operates at three levels. Each level runs independently. When task evaluation identifies deviated prompts, they can be fed into prompt-level analysis for deeper investigation.

Level 1: Task-level validation

Run dataset evaluations (lm_eval, longbench) against a vLLM server and compare aggregate scores against user-defined thresholds.

- Run accuracy benchmarks (e.g., GSM8K, MMLU Pro, BBH).
- Compare scores against thresholds (simple minimum or mean/std tolerance).
- Report pass/fail with per-dataset scores.

Pass/fail criteria: Accuracy scores must meet thresholds defined by the user.

Level 2: Prompt-level validation

Validate model outputs at the token level using pre-defined or deviated prompts. For each prompt, compute HF reference logits (FP32 + BF16) and run token-by-token comparison with teacher forcing.

- **Logit validation** — Three-way comparison (HF FP32 vs HF BF16 vs Neuron) using top-k error maps at k={5, 50, 1000, all} and divergence detection.
- **KV cache analysis** — Three-way comparison of KV caches with per-layer, per-head error metrics and Bhattacharyya Coefficient (BC) to classify errors as BF16-inherent vs anomalous.

Pass/fail criteria: Logit divergence within tolerance maps; KV cache BC ≥ 0.99.

Level 3: Module-level validation

Per-module correctness tests validate individual model components (attention, MLP, RMSNorm, embedding, RoPE, decoder layer) against HuggingFace reference implementations. These run in CPU mode and on hardware.

Pass/fail criteria: Output tensors match HF reference within tolerance.

Tip: Start at Level 1 to confirm there is a real accuracy issue, then use Level 2 to isolate which tokens and layers diverge, and Level 3 to pinpoint the exact module.

Debugging instructions

The following steps walk through a systematic workflow for isolating accuracy issues, from ruling out false alarms to pinpointing the divergent layer:

1. Rule out sampling variance
2. Reproduce with a minimal, deterministic prompt
3. Compare outputs against a reference
4. Isolate the introduction point with a layered check
5. Interpret the divergence pattern
6. Check for common causes
7. Apply a fix and validate

Step 1: Rule out sampling variance

Before investigating hardware or framework issues, confirm that the difference you observe is not normal sampling variance. Many false alarms come from non-deterministic generation settings.

Set your generation to be fully deterministic:

```
# Use greedy decoding to eliminate sampling randomness
sampling_params = SamplingParams(
    temperature=0.0,  # greedy
    max_tokens=64,    # short output for fast iteration
)
```

If your issue disappears under greedy decoding, the problem is likely in your sampling configuration (temperature, top-p, top-k) rather than in the model computation. Compare the sampling parameters between your Neuron deployment and your reference environment before proceeding.

Note: Even with greedy decoding, BF16 arithmetic can produce different top-1 tokens than FP32 when the top two logits are very close. This is expected behavior, not a bug. If only a small fraction of tokens differ and the overall quality is acceptable, you may not have an accuracy issue.

Step 2: Reproduce the issue with a minimal, deterministic prompt

Reduce your reproduction to the smallest possible case:

1. Select a single prompt that demonstrates the issue.
2. Set a fixed random seed if your configuration uses one.
3. Use greedy decoding (`temperature=0.0`).
4. Limit `max_tokens` to the shortest length that still shows the divergence.

```
# Minimal reproduction setup
prompt = "The capital of France is"  # Use your failing prompt
sampling_params = SamplingParams(temperature=0.0, max_tokens=32)

# Generate on Neuron
neuron_output = llm.generate([prompt], sampling_params)

# Compare against reference (e.g., HuggingFace on CPU)
# hf_output = run_hf_reference(prompt, max_tokens=32)
```

A minimal reproduction makes the remaining steps faster and eliminates confounding variables like batch interactions or long-context effects.

Step 3: Compare outputs against a reference

Start at the coarsest level and narrow down:

1. **Full text comparison** — Do the generated strings differ? At which token position does the first divergence occur?
2. **Top-1 token comparison** — At the first divergent position, what are the top-1 tokens from each environment?
3. **Logit distribution comparison** — Compare the full logit vectors at the divergent position. Are the top-k rankings similar with only small magnitude differences, or is the distribution fundamentally different?

The position of the first divergent token tells you where to focus:

- **Token 0 diverges** — The issue is in prefill (the first forward pass). Check weight loading, embedding, and the prefill attention computation.
- **Token 1+ diverges, token 0 matches** — The issue is in decode. Check the KV cache, decode attention, or position encoding for generated positions.

Tip: The logit validation tool automates this comparison using teacher forcing (forcing ground-truth token when it diverges to isolate per-position errors from autoregressive drift).

Step 4: Isolate the introduction point with a layered check

Work through the model pipeline in order, checking each stage against your reference. Stop at the first stage that shows unexpected divergence:

#	Layer	What to check
1	Tokenizer	Confirm both environments produce identical token IDs for your prompt. Tokenizer version mismatches are a common source of apparent “accuracy” issues.
2	Weight loading	Verify the checkpoint loaded correctly. Compare a sample of weight tensors between Neuron and your reference. Check for dtype mismatches (FP32 vs BF16) or truncated/corrupted weights.
3	Embedding output	Compare the embedding layer output for your prompt tokens. This isolates weight-loading issues from computation issues.
4	Attention output	Compare attention outputs at one or more decoder layers. Differences here point to attention implementation issues (RoPE encoding, attention mask, head layout).
5	KV cache	Compare cached key/value tensors against reference. Issues here cause decode-phase divergence even when prefill is correct.
6	Post-quantization output	If using quantization, compare the quantized model output against the BF16 baseline. Quantization drift is expected but should be bounded.
7	Sampling / logit processing	Compare raw logits before and after any logit processors (repetition penalty, temperature scaling). Confirm the sampling implementation matches your reference.

Note: The logit validation tool performs a three-way comparison (HF FP32 vs HF BF16 vs Neuron) at the logit and KV cache levels. This distinguishes BF16-inherent numerical noise from vLLM-on-Neuron-specific bugs.

Step 5: Interpret the divergence pattern

Use the pattern of divergence to classify the root cause:

Pattern	Interpretation
Neuron-vs-FP32 error is similar in magnitude to BF16-vs-FP32 error (ratio ≈ 1.0)	The divergence is BF16 numerical noise, not a vLLM-on-Neuron bug. This is expected behavior.
Neuron-vs-FP32 error is much larger than BF16-vs-FP32 error (ratio >> 1.0)	A vLLM-on-Neuron-specific issue exists. Proceed to isolate the specific layer or component.
Token 0 fails, subsequent tokens also fail	Prefill bug. The error propagates from the first forward pass. Check weight loading and prefill attention.
Token 0 passes, tokens 1+ fail	Decode-phase bug. Check KV cache write/read, decode attention, or position encoding at generated positions.
Errors appear only at specific sequence positions	Likely a position encoding or attention mask issue. Check RoPE implementation and context length handling.

Step 6: Check for common causes

Review the following common causes of accuracy drift. For each, the symptom pattern and where to look:

Wrong checkpoint format or incomplete weight loading

Symptom: Large divergence starting at token 0 across all layers. Output may be nonsensical.

Where to check: Compare weight tensor shapes and values between your loaded model and the original checkpoint. Look for warnings during model loading about missing or unexpected keys. Verify the checkpoint format (safetensors vs bin) matches what the loader expects.

Dtype mismatch between reference and deployment

Symptom: Small but consistent divergence across all tokens. Three-way comparison shows Neuron error is proportional to BF16 error.

Where to check: Confirm both environments use the same dtype. A common mistake is comparing FP32 reference outputs against BF16 Neuron outputs and interpreting the BF16 rounding as a bug. Always compare BF16-to-BF16 when assessing Neuron-specific accuracy.

Quantization-induced drift exceeding tolerance

Symptom: Moderate divergence that scales with quantization aggressiveness. Output is coherent but subtly different from the BF16 baseline.

Where to check: Compare BF16 Neuron output against quantized Neuron output to isolate the quantization contribution. If the drift is within your quality tolerance, it is working as designed. If not, consider a less aggressive quantization scheme or a different granularity (per-channel vs per-tensor).

KV cache inconsistency under prefix caching or paged attention

Symptom: Outputs differ depending on whether a prefix cache hit occurs. The same prompt produces different results on first vs subsequent runs, or results change when `block_size` changes.

Where to check: Disable prefix caching and compare. If the issue disappears, check that `block_size` and `max_model_len` are compatible with your prompt lengths. Verify that KV cache reconstruction from paged blocks produces the same values as contiguous computation.

Speculative decoding verification failure

Symptom: Occasional incorrect tokens that do not appear under standard (non-speculative) decoding.

Where to check: Disable speculative decoding and compare. If the issue resolves, the draft model may be proposing tokens that the verification step incorrectly accepts. Check that the draft and target models use compatible tokenizers and that the verification threshold is correctly configured.

Unsupported feature combination

Symptom: Incorrect output that correlates with enabling a specific feature (e.g., a particular attention variant, context length extension, or parallelism configuration).

Where to check: Consult the Reference: vLLM on Neuron feature and model compatibility matrix to confirm your feature combination is supported. Some combinations are not yet validated and may produce incorrect results silently.

Step 7: Apply a fix and validate

After identifying the root cause, apply your fix and validate systematically:

1. **Rerun the minimal prompt** — Confirm the fix resolves the original failing case under greedy decoding.
2. **Test additional prompts** — Run 5–10 diverse prompts that previously showed divergence.
3. **Run a broader evaluation** — Use a standard benchmark (e.g., GSM8K, MMLU) to confirm the fix does not introduce regressions elsewhere.
4. **Compare against your quality threshold** — Define an acceptable accuracy tolerance and verify the deployment meets it.

```
# Example: run lm_eval to validate accuracy after a fix
lm_eval --model vllm \
    --model_args pretrained=/path/to/model,tensor_parallel_size=8 \
    --tasks gsm8k \
    --batch_size auto
```

Important

A fix that resolves one prompt but degrades others is not a fix. Always validate broadly before declaring the issue resolved.

Confirm your work

To confirm the accuracy issue is resolved:

1. The minimal reproduction prompt now produces output matching your reference under greedy decoding.
2. All inspected layers report divergence within the expected BF16 noise range (three-way ratio ≈ 1.0).
3. A broader evaluation (multiple prompts or a benchmark) shows scores within your defined tolerance of the reference.

Common issues

Tokenizer mismatch between reference and deployment

- **Possible solution:** Confirm both environments load the same tokenizer files and version. Small tokenizer updates can produce different token IDs for the same text, causing apparent model divergence. Always load the tokenizer from the same checkpoint path in both environments.

Quantization-induced drift exceeds quality threshold

- **Possible solution:** Compare BF16 versus quantized outputs to quantify the quantization contribution separately from other issues. If drift exceeds your tolerance, try a different quantization scheme (e.g., switch from W8A8 to W8A16) or increase quantization granularity (per-channel instead of per-tensor).

Prefix caching produces different output than non-cached runs

- **Possible solution:** This indicates a cache consistency issue. Verify that `block_size`, `max_model_len`, and `num_gpu_blocks_override` are compatible with your prompt length distribution. As a diagnostic step, disable prefix caching to confirm it is the source.

Output looks correct for short prompts but degrades at longer context

- **Possible solution:** Check that the model's maximum context length configuration matches the checkpoint's trained context length. Verify that RoPE scaling parameters (if used) are correctly configured. Compare attention outputs at positions near the context boundary.

Accuracy differs between tensor parallelism degrees

- **Possible solution:** Some numerical differences across TP degrees are expected due to reduction order. If the differences are large, check that the weight sharding is correct for your TP configuration and that the model architecture supports the TP degree you selected.

Accuracy tools reference

vLLM on Neuron provides a suite of accuracy debugging tools. Each tool addresses a different level of the debugging workflow.

Logit validation

Compares raw logits between a reference model (HF) and the target model (vLLM on Neuron) using teacher forcing. Teacher forcing forces ground-truth token when it diverges, isolating per-position errors from autoregressive drift. Supports three-way comparison (HF FP32 vs HF BF16 vs Neuron) to distinguish BF16-inherent numerical noise from vLLM-on-Neuron-specific bugs.

API:

```
from vllm_neuron.accuracy.logit_validation import (
    logit_validation,
    multi_prompt_logit_validation,
)

# Single prompt
passed = logit_validation(
    input_ids=input_ids,
    generate_fn=neuron_generate_fn,
    expected_logits=reference_logits,
    visualize=True,
    output_dir="validation_results",
)

# Multiple prompts
result = multi_prompt_logit_validation(
    prompts_input_ids=prompts_input_ids,
    generate_fn=neuron_generate_fn,
    prompts_expected_logits=expected_logits,
    prompts_baseline_logits=baseline_logits,  # optional FP32 for three-way
)
```

Tolerance configuration:

Divergence is measured at multiple top-k levels with configurable thresholds:

```
DEFAULT_TOLERANCE_MAP = {
    "5": (1e-5, 0.01),
    "50": (1e-5, 0.02),
    "1000": (1e-5, 0.03),
    "all": (1e-5, 0.05),
}
```

Output: Interactive HTML report (`logit_analysis_b{batch}.html`) with four subgraphs per prompt:

- **Error Map (top)** — Tracks divergence values across top-k tokens for different k values (k=5, k=50, k=1000, k=all). The divergence shows the relative error between target and reference logits. Token divergences are marked as vertical orange lines. X-axis: token position; Y-axis: divergence value.
- **Relative Difference (middle-top)** — Measures the Top1 logit minus Top2 logit difference for both expected and actual models. When this difference is smaller than the unit of least precision (ULP), it could lead to wrong token selection due to precision limits. X-axis: token position; Y-axis: relative difference (always > 0).
- **Top1 Indices Comparison (middle-bottom)** — Compares which tokens are predicted as most likely between models. Expected (cyan) vs actual (pink) top-1 token indices. Diverging lines indicate different token selection. X-axis: token position; Y-axis: token index.
- **Top2 Logit Values (bottom)** — Shows the actual logit values for the top-2 predictions. Values should match between expected and actual. Helps validate numerical precision across platforms. X-axis: token position; Y-axis: logit value.

KV cache analysis

Compares KV caches between a reference model and vLLM on Neuron to debug accuracy issues at the attention layer level. Supports two-way and three-way comparison.

API:

```
from vllm_neuron.accuracy.kv_cache_analysis import (
    extract_hf_kv_caches_teacher_forced,
    extract_vllm_kv_cache_config,
    extract_vllm_kv_caches,
    extract_vllm_block_tables,
    reconstruct_contiguous_kv,
    compare_kv_caches,
    print_kv_report,
)

# Extract HF reference KV caches (teacher-forced)
hf_kv = extract_hf_kv_caches_teacher_forced(model, input_ids, teacher_tokens)

# Extract vLLM paged KV caches
kv_config = extract_vllm_kv_cache_config(llm)
paged_kv = extract_vllm_kv_caches(llm, kv_config)
block_tables = extract_vllm_block_tables(llm)

# Reconstruct contiguous KV from paged format
vllm_kv = reconstruct_contiguous_kv(paged_kv, kv_config, block_tables, seq_len)

# Compare (two-way or three-way)
result = compare_kv_caches(expected_kv=hf_kv, actual_kv=vllm_kv)
result = compare_kv_caches(
    expected_kv=hf_kv, actual_kv=vllm_kv,
    baseline_kv=fp32_kv)
print_kv_report(result)
```

Metrics per token, per layer, per head:

- Cosine similarity (1.0 = identical)
- Relative L-inf: `max|diff| / max|ref|`
- Relative L2: `||diff||_2 / ||ref||_2`
- Bhattacharyya Coefficient (BC): overlap between expected and actual error distributions (three-way only). BC ≈ 1.0 means errors match dtype-only noise; BC → 0 means anomalous divergence.

Tensor capture

Extracts intermediate tensor values from compiled models for accuracy debugging. Enables comparison to identify which module first diverges.

Configuration via vLLM:

```
from vllm import LLM

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    additional_config={
        "neuron_config": {
            "tensor_capture": {
                "modules": ["model.layers.0-31", "lm_head"],
                "capture_dir": "/tmp/captures",
            }
        }
    },
)
```

Programmatic API:

```
from vllm_neuron.accuracy import TensorCaptureModel, CaptureWriter
```

```
# Wrap model before compile
capture_model = TensorCaptureModel(model, ["model.layers.0-31"])
compiled = torch.compile(capture_model, backend="vllm_neuron", fullgraph=True)

# Create writer after compile, enable after warmup
writer = CaptureWriter("/tmp/captures", dp_rank=0, tp_rank=0)
writer.enable()
```

Manual capture inside model code:

```
from vllm_neuron.accuracy import capture_tensor

class LlamaAttention(nn.Module):
    def forward(self, hidden_states, ...):
        attn_output = self._compute_attention(...)
        capture_tensor("attn_output", attn_output)
        return attn_output
```

Pattern syntax: Range expansion (`model.layers.0-31`) and Python regex (`.*self_attn$`).

Output structure:

```
/tmp/captures/
■■■ dp0/
■   ■■■ prefill_s128_0/
■   ■   ■■■ model.layers.0/
■   ■   ■   ■■■ rank0.pt
■   ■   ■■■ prefill_s128_0_meta.json
■   ■■■ decode_b1_0/
■       ■■■ model.layers.0/
■       ■   ■■■ rank0.pt
■       ■■■ decode_b1_0_meta.json
```

Tensor compare

Compares captured tensors between different execution environments (CPU vs Neuron, FP32 vs BF16) to identify numerical divergence.

Two-way comparison (direct comparison with static threshold):

```
from vllm_neuron.accuracy import compare_tensors, compare_capture_dirs

# Single tensor
result = compare_tensors(cpu_tensor, neuron_tensor, name="layer_0")
# result.linf_rel, result.l2_rel, result.max_abs

# Compare full capture directories
results = compare_capture_dirs("/tmp/captures/cpu", "/tmp/captures/neuron")
print_comparison_report(results, threshold=0.01)
```

Three-way comparison (dynamic threshold from FP32 baseline):

```
from vllm_neuron.accuracy import compare_tensors_three_way

result = compare_tensors_three_way(
    baseline=fp32_tensor,       # FP32 ground truth
    expected=bf16_cpu_tensor,   # BF16 on CPU (dtype reference)
    actual=bf16_neuron_tensor,  # BF16 on Neuron (target)
    name="layer_0",
)
# result.linf_ratio, result.l2_ratio, result.bc, result.passed
```

Three-way comparison answers: “Is this error from Neuron, or just from BF16?” If `linf_ratio` ≈ `1.0` and `bc` ≥ `0.99`, the error is dtype-inherent. If `linf_ratio >> 1.0`, there is a target-specific issue.

Module-level testing

Per-module correctness tests validate individual model components (attention, MLP, RMSNorm, embedding, RoPE, decoder layer) against HuggingFace reference implementations.

Two-way assertion (fast iteration with known threshold):

```
from vllm_neuron.accuracy.testing import assert_close

assert_close(
    neuron_output, fp32_output,
    rtol=1e-2, atol=1e-2, name="attn_prefill")
```

Three-way assertion (statistical accuracy gate):

```
from vllm_neuron.accuracy.testing import assert_close_three_way

assert_close_three_way(
    hf_output_fp32,     # FP32 baseline
    hf_output_bf16,     # BF16 on CPU
    neuron_output,      # BF16 on Neuron
    name="attn_prefill/layer0/seq128/tp8",
)
```

`assert_close` normalizes by the global maximum of the expected tensor (not per-element), avoiding false failures on near-zero elements. On failure, it reports mismatch count, max relative error, and max absolute error with element indices.

Staged enablement: Module → N-Layer → Full Model → E2E Eval. Each stage validates accuracy before proceeding.

Related information

- Reference: vLLM on Neuron feature and model compatibility matrix — Supported feature and model combinations.
- How to use vLLM on Neuron features in your deployment — Feature configuration options that can affect accuracy.
- How to onboard a model to vLLM on Neuron — Model onboarding flow, which includes an accuracy validation step.

How to deploy gpt-oss with vLLM on Neuron

Task overview

This topic discusses how to deploy gpt-oss with vLLM on Neuron as a production-ready recipe. It covers **gpt-oss 120B on Trn3** and **gpt-oss 20B on Trn2 or Trn3**, on a single instance with tensor parallelism. The recipe assumes you have already worked through How to set up vLLM on Neuron and one of the serving quickstarts.

Tested versions

This recipe was validated against the following components. If you are on a newer release, confirm the parameter set still applies before promoting it to production.

Component	Version
Neuron SDK	2.30
vLLM-Neuron plugin	Shipped with the Neuron 2.30 release
vLLM (upstream)	The version pinned by the vLLM-Neuron plugin for Neuron 2.30
gpt-oss 120B checkpoint	`openai/gpt-oss-120b` (Hugging Face)

gpt-oss 20B checkpoint	`openai/gpt-oss-20b` (Hugging Face)

Target configuration

The recipe runs both gpt-oss 120B and gpt-oss 20B with the same parameter set. The only deployment-time choices are which model checkpoint you serve and which instance you serve it on.

Supported instance and quantization combinations:

- **gpt-oss 120B:** Trn3 only. MXFP4-quantized weights are auto-selected on Trn3 from the published `openai/gpt-oss-120b` checkpoint.
- **gpt-oss 20B on Trn3:** MXFP4 weights are auto-selected from the published `openai/gpt-oss-20b` checkpoint.
- **gpt-oss 20B on Trn2:** Runs in BF16. MXFP4 is Trn3-only.

Recipe parameters:

Setting	Value	Notes
Tensor parallelism	8	Same TP for both 120B and 20B in this recipe.
`max_num_seqs`	2	Conservative starting point; raise only after validating headroom.
`max_model_len`	Model default (131072)	The launch command in this recipe does not override `--max-model-len`. If you hit out-of-memory at startup, set it explicitly to a smaller value (for example, `--max-model-len 4096`) and pass `--max-num-batched-tokens` to match.
Quantization	MXFP4 on Trn3, BF16 on Trn2	Always pass `--hf-overrides '{"quantization_config": {}}'` so vLLM defers quantization handling to the Neuron backend, which then auto-selects MXFP4 on Trn3.
Prefix caching	On (default)	vLLM enables prefix caching by default.
Async scheduling	On (default)	vLLM-Neuron enables async scheduling by default; this recipe does not override it.
On-device sampling	top-k (default)	The Neuron backend uses on-device top-k sampling by default. Set `all_greedy: true` if you need deterministic outputs (see step 4).
Prefill segmentation	On (default)	vLLM-Neuron uses prefill segmentation to split long prompts into pure-prefill executions. The segment size is controlled by `num_batched_tokens_buckets` in `neuron_config` (see step 4).

Prerequisites

- **vLLM on Neuron environment:** A working vLLM on Neuron setup on the target instance. See How to set up vLLM on Neuron.
- **Model access:** Access to the gpt-oss checkpoint you intend to serve.
- **Disk for the checkpoint:** gpt-oss 120B is a large download. Provide a fast local SSD path via vLLM's `--download-dir` or pre-stage the checkpoint to local storage.

- **Familiarity with vLLM serving:** See Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron and Quickstart: Run offline inference with vLLM on Neuron.

Instructions

1: Set environment variables for compilation and runtime

gpt-oss compilation can run longer than the vLLM defaults. Raise the timeouts before starting the server.

```
# Compilation and execution timeouts (gpt-oss compiles can exceed default 600s).
export VLLM_NEURON_COMPILATION_TIMEOUT=1200
export VLLM_EXECUTE_MODEL_TIMEOUT_SECONDS=1200
```

2: Launch the vLLM server

Run this on the target instance. The command below is the centerpiece configuration. It uses vLLM defaults for prefix caching, async scheduling, on-device sampling, and `max_model_len`, and only sets the parameters that gpt-oss requires.

```
vllm serve openai/gpt-oss-120b \
    --tensor-parallel-size 8 \
    --max-num-seqs 2 \
    --hf-overrides '{"quantization_config": {}}'
```

To deploy gpt-oss 20B instead, swap only the model identifier:

```
vllm serve openai/gpt-oss-20b \
    --tensor-parallel-size 8 \
    --max-num-seqs 2 \
    --hf-overrides '{"quantization_config": {}}'
```

Note

On the first launch the server compiles the model graphs before it accepts traffic. Expect several minutes of compile time. Subsequent launches reuse cached NEFFs unless you explicitly invalidate the cache. See How to set up vLLM on Neuron for cache configuration.

3: Understand the gpt-oss-specific flags

A few flags warrant explicit explanation because they are gpt-oss specific or because the defaults are not what you want for this recipe.

- `--hf-overrides '{"quantization_config": {}}'` — gpt-oss ships with a `quantization_config` block in its Hugging Face config that vLLM would otherwise interpret. Overriding it with an empty object lets the Neuron backend handle the model's weights through its own loader. **Omit this flag and the server fails to load the model.**
- `--tensor-parallel-size 8` — Sized to fit weights and KV cache in HBM with the default `max_model_len`.
- `--max-num-seqs 2` — Conservative starting point. Raise only after you have validated headroom on your instance.

4: (Optional) Tune sampling and prefill segmentation

If the defaults do not match your workload, the Neuron backend exposes additional knobs through `--additional-config`. The two you are most likely to set for gpt-oss are:

- **``on_device_sampling_config.all_greedy: true``** — Switch on-device sampling from the default top-k path to greedy. Use for deterministic outputs (for example, regression tests or accuracy comparisons).
- **``num_batched_tokens_buckets``** — Sets the prefill segmentation size used to split long prompts into pure-prefill executions. The default is fine for most workloads. Pin it to a single value when you want a single, predictable compiled graph. If you set it, the last bucket value must match `--max-num-batched-tokens`.

Example setting both knobs — greedy on-device sampling and a 4096-token prefill segment — while keeping `max_model_len` at its default:

```
vllm serve openai/gpt-oss-120b \
    --tensor-parallel-size 8 \
    --max-num-seqs 2 \
    --max-num-batched-tokens 4096 \
    --hf-overrides '{"quantization_config": {}}' \
    --additional-config '{
        "neuron_config": {
            "num_batched_tokens_buckets": [4096],
            "on_device_sampling_config": {"all_greedy": true}
        }
    }'
```

`--max-num-batched-tokens` is set so it matches the last bucket value, which is the bucket validator's requirement. `max_model_len` remains at the model default, so prompts longer than 4096 tokens are split into 4096-token segments at prefill.

5: Validate the deployment

With the server running, send a representative request and confirm the response shape.

```
# Health check
curl -i http://localhost:8000/health
```

You should see `HTTP/1.1 200 OK`.

Next, send a sample completion request. Replace `openai/gpt-oss-120b` with `openai/gpt-oss-20b` if you launched the 20B variant.

```
curl http://localhost:8000/v1/completions \
    -H "Content-Type: application/json" \
    -d '{
        "model": "openai/gpt-oss-120b",
        "prompt": "The capital of France is ",
        "max_tokens": 16
    }'
```

The response is an OpenAI-compatible JSON payload. Confirm the following:

- `choices[0].text` contains a coherent continuation of the prompt.
- `choices[0].finish_reason` is `length` (because the request hit `max_tokens`) or `stop`.
- `usage.prompt_tokens` and `usage.completion_tokens` are populated.

Note

With default top-k sampling, completion content varies across runs. For deterministic outputs, enable greedy on-device sampling as shown in step 4. For substantive accuracy validation, see How to debug accuracy issues with vLLM on Neuron and **`../how-to-use-accuracy-debugger`**.

6: Benchmark and tune prefix caching

Before promoting this configuration to production, run a workload-shaped benchmark to quantify the throughput and time-to-first-token impact of prefix caching for your prompt distribution.

Confirm your work

You have a successful deployment when:

- The server starts without errors and the `/health` endpoint returns 200.
- `/v1/completions` returns an OpenAI-compatible response on the validation prompt with the expected shape.

- A short representative-traffic run produces stable, on-spec completions.
- Throughput and latency on your benchmark traffic meet the targets you set for the instance.

Common issues

Server fails to load gpt-oss because of `quantization_config`

- **Possible solution:** Confirm you passed `--hf-overrides '{"quantization_config": {}}'`. Without that override, vLLM tries to apply the model's published quantization config and fails before the Neuron backend gets the weights.

Compilation hits the timeout

- **Possible solution:** gpt-oss is large; compilation can exceed the default 600-second budget. Set `VLLM_NEURON_COMPILATION_TIMEOUT=1200` (or higher) before launch. If compilation continues to time out, see How to set up vLLM on Neuron for cache configuration and shared-cache options.

Out-of-memory at startup

- **Possible solution:** The model default `max_model_len` is 131072, which can exceed available HBM for gpt-oss at TP=8. Pass an explicit `--max-model-len` (for example, `4096`) and a matching `--max-num-batched-tokens`. Then reduce `--max-num-seqs` if you still hit the limit.

Related information

- How to use vLLM on Neuron features in your deployment — Feature configuration for gpt-oss and other models.
- Reference: vLLM on Neuron feature and model compatibility matrix — Confirm which features apply to gpt-oss in the current release.
- Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron — Underlying online serving flow.
- Quickstart: Run offline inference with vLLM on Neuron — Equivalent offline batch flow.
- How to debug accuracy issues with vLLM on Neuron — Investigate accuracy issues if validation completions look wrong.

Tutorial: Deploy EAGLE3 speculative decoding with vLLM on Neuron

This tutorial guides you through deploying EAGLE3 speculative decoding with vLLM on Neuron using a public EAGLE3 draft checkpoint from Hugging Face. When you have completed it, you will have a vLLM on Neuron server running Llama 3.1 8B Instruct accelerated with an EAGLE3 draft model, observed the draft acceptance rate, and measured the throughput improvement over the non-speculative baseline on the public `sonnet` dataset.

Overview

Speculative decoding uses a small *draft* model to propose several candidate tokens per step. The larger *target* model then verifies all proposals in a single forward pass. Each accepted proposal yields an extra token from one target step, lifting throughput without changing the target model's output.

EAGLE3 (arxiv.org/abs/2503.01840) is a draft architecture that conditions on hidden states from the target model. Because it can reuse the target's intermediate state, EAGLE3 typically achieves higher acceptance rates than a vanilla draft-model approach in which an independently trained smaller model (for example, Llama 3.2 1B as a draft for Llama 3.1 8B) proposes tokens. vLLM on Neuron supports EAGLE3 natively via the `--speculative-config` flag.

This tutorial uses the following target-draft model pair:

- **Target:** meta-llama/Llama-3.1-8B-Instruct
- **Draft:** RedHatAI/Llama-3.1-8B-Instruct-speculator.eagle3

Before you start

This tutorial assumes that you have experience in the following areas:

- Running a vLLM on Neuron server. See Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron.
- Working with Hugging Face models and access tokens.
- Basic familiarity with speculative decoding concepts.

Model details

This tutorial uses Llama 3.1 8B Instruct with the following settings:

- `tensor-parallel-size 8`
- `max-num-seqs 2`
- `max-model-len 2048`
- `num_speculative_tokens 3`

Limitations:

- The server is configured for a decode concurrency of 2. Higher values require additional decode batch-bucket coverage and are out of scope for this tutorial.
- If the implementation of either the target model or the draft model is not accurate, the acceptance rate will be low and speculative decoding will add compute overhead instead of boosting decode throughput.

Prerequisites

- `trn2.48xlarge` instance with Neuron SDK 2.30 or later. See How to set up vLLM on Neuron.
- vLLM-Neuron plugin installed.
- Hugging Face account that has accepted the Llama 3.1 license at https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct.
- Hugging Face access token with read access to public gated repositories. Fine-grained tokens that lack this permission return HTTP 403 on gated repos even when the license is accepted.

Prepare your environment

Set your Hugging Face access token in the shell you will use to launch the server. You do not need to pre-download the checkpoints — vLLM resolves both model IDs and downloads them on first launch.

```
export HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Verify the token has access to the gated target repository. The script below pulls only the small `config.json` to confirm authentication; the full model and draft checkpoints are downloaded by `vllm serve` on first launch.

```
python - <<'PY'
from huggingface_hub import hf_hub_download
hf_hub_download(
    repo_id="meta-llama/Llama-3.1-8B-Instruct",
    filename="config.json",
)
print("OK")
PY
```

If the script prints `OK`, you are ready. If it returns `403 Forbidden`, update your token permissions at https://huggingface.co/settings/tokens and retry.

Download the public `sonnet` dataset, used for the throughput benchmarks in Step 1 and Step 2:

```
wget -O sonnet.txt \
    https://raw.githubusercontent.com/vllm-project/vllm/main/benchmarks/sonnet.txt
```

Step 1: Run the non-speculative baseline

In this step, you will launch Llama 3.1 8B Instruct without speculative decoding, smoke-test it, and benchmark it. You will use this baseline to measure the throughput improvement EAGLE3 delivers in Step 2.

Launch the baseline server:

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --max-num-seqs 2 \
    --max-model-len 2048 \
    --no-enable-prefix-caching \
    --port 8000
```

First-run compilation and warmup take several minutes. Wait until the server log prints `INFO: Application startup complete.` before continuing.

Confirm the server responds:

```
curl http://localhost:8000/v1/completions \
    -H "Content-Type: application/json" \
    -d '{
      "model": "meta-llama/Llama-3.1-8B-Instruct",
      "prompt": "I am gonna keep counting forever, 1 2 3 4 5 ",
      "max_tokens": 10,
      "temperature": 0
    }'
```

The `text` field should continue the counting sequence (for example, `"6 7 8 9 10 "`).

Run the baseline benchmark:

```
vllm bench serve \
    --base-url http://localhost:8000 \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --dataset-name sonnet \
    --dataset-path ./sonnet.txt \
    --sonnet-input-len 512 \
    --sonnet-output-len 128 \
    --num-prompts 50 \
    --max-concurrency 2 \
    --save-result \
    --result-filename baseline.json
```

When the benchmark finishes, stop the baseline server with `Ctrl+C`. Both servers require all 8 Neuron devices for `--tensor-parallel-size 8`, so the EAGLE3 server in Step 2 cannot run alongside it.

Step 2: Run EAGLE3 and compare

In this step, you will launch the EAGLE3 server, smoke-test it, benchmark it on the same dataset, and compare the results against the baseline from Step 1.

Launch the EAGLE3 server:

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --tensor-parallel-size 8 \
    --max-num-seqs 2 \
    --max-model-len 2048 \
    --no-enable-prefix-caching \
    --speculative-config '{
        "method": "eagle3",
```

```
        "model": "RedHatAI/Llama-3.1-8B-Instruct-speculator.eagle3",
        "num_speculative_tokens": 3
      }' \
    --port 8000
```

The new flag is `--speculative-config`:

- `method: eagle3` — selects the EAGLE3 draft architecture.
- `model` — draft checkpoint, which must be trained against this specific target.
- `num_speculative_tokens: 3` — number of tokens the draft proposes per target step.

First-run compilation takes longer than the baseline because both the target and the draft model compile. Wait again for `INFO: Application startup complete.`.

Confirm the server responds. Under greedy sampling, EAGLE3 is a lossless acceleration: the target model's output is unchanged regardless of what the draft proposes, so the response should match the baseline output from Step 1.

```
curl http://localhost:8000/v1/completions \
    -H "Content-Type: application/json" \
    -d '{
      "model": "meta-llama/Llama-3.1-8B-Instruct",
      "prompt": "I am gonna keep counting forever, 1 2 3 4 5 ",
      "max_tokens": 10,
      "temperature": 0
    }'
```

Run the EAGLE3 benchmark:

```
vllm bench serve \
    --base-url http://localhost:8000 \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --dataset-name sonnet \
    --dataset-path ./sonnet.txt \
    --sonnet-input-len 512 \
    --sonnet-output-len 128 \
    --num-prompts 50 \
    --max-concurrency 2 \
    --save-result \
    --result-filename eagle3.json
```

When the benchmark runs against an EAGLE3 server, `vllm bench serve` prints a Speculative Decoding block with the draft acceptance metrics. This is the headline signal for whether EAGLE3 is helping on your workload — the higher the acceptance rate, the more tokens each target step produces.

```
---------------Speculative Decoding---------------
Acceptance rate (%):                    74.14
Acceptance length:                      3.22
Drafts:                                 1976
Draft tokens:                           5928
Accepted tokens:                        4395
Per-position acceptance (%):
  Position 0:                           86.49
  Position 1:                           72.47
  Position 2:                           63.46
```

Key fields:

- `Acceptance rate` — overall fraction of drafted tokens that were accepted.
- `Acceptance length` — average tokens produced per target step. `1.0` means no draft tokens were accepted (equivalent to no speculation). `num_speculative_tokens + 1` is the theoretical maximum.
- `Per-position acceptance` — acceptance per draft position. The first position is usually the highest; a steep drop-off suggests the draft is only useful for the immediate next token.

Compare the two benchmark runs:

```
python - <<'PY'
import json
b = json.load(open("baseline.json"))
e = json.load(open("eagle3.json"))
for k in ("output_throughput", "median_tpot_ms", "median_ttft_ms"):
    print(f"{k:20s}  baseline={b[k]:.2f}  eagle3={e[k]:.2f}")
speedup = e['output_throughput'] / b['output_throughput']
print(f"\nEAGLE3 output throughput speedup: {speedup:.2f}x")
PY
```

EAGLE3 should report higher `output_throughput` and lower `median_tpot_ms` than the baseline. Time-to-first-token is slightly higher under EAGLE3 because the draft model adds a small amount of compute before the first token; this is expected and is more than offset by the per-token speedup.

Step 3: Tune for your workload

The two settings that have the biggest impact are the draft/target pair and `num_speculative_tokens`.

- **Confirm that both draft and target models are accurate.** No tuning compensates for an inaccurate model on either side. If Step 2 shows an acceptance rate below roughly 30%, stop tuning and validate the draft and target implementations against a known reference before continuing.
- **``num_speculative_tokens``.** Higher values propose more tokens per step. They yield more when acceptance is high and lose more when acceptance is low, because rejected drafts still consume compute. Sweep 2, 3, and 4 on representative traffic and pick the value with the highest `output_throughput` — not the highest acceptance rate.
- **Workload alignment.** EAGLE3 acceptance depends on the datasets the draft head was trained on. Acceptance rates degrade when the request distribution at serving time diverges from the draft's training distribution. If your traffic is far from the draft's training mix, expect lower speedups and consider a draft trained on data closer to your workload.

Confirmation

You have launched a baseline server and an EAGLE3 server, verified they produce equivalent greedy output, observed the draft acceptance rate, and measured the throughput improvement on the sonnet dataset. If you encountered any issues, see the **Common issues** section below.

Common issues

- **Acceptance rate is 0% or near 0%.** If the implementation of either the target or the draft is not accurate, acceptance collapses. Confirm both models are accurate against a known reference and that the draft checkpoint was trained against this exact target.
- **HTTP 403 when the server pulls the model.** Your Hugging Face token lacks the "read access to public gated repositories" permission, or the account has not accepted the Llama 3.1 license. Update the token at https://huggingface.co/settings/tokens, accept the license at https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct, and restart the server.
- **Throughput regressed with EAGLE3 enabled.** The EAGLE3 acceptance rate is not high enough to compensate for the speculative decoding compute overhead. Speculative decoding pays compute for every draft token and recovers it only on accepted tokens. Check Step 2's Speculative Decoding block; if acceptance is low, reduce `num_speculative_tokens` or use a draft trained on data closer to your serving distribution.

Clean up

Stop the vLLM servers with `Ctrl+C`. If you launched an EC2 instance for this tutorial, terminate it to avoid ongoing charges.

Next steps

- How to use vLLM on Neuron features in your deployment — Other features you can stack with speculative decoding.
- Reference: vLLM on Neuron feature and model compatibility matrix — Confirm which target models support speculative decoding on vLLM on Neuron.
- Quickstart: Run offline inference with vLLM on Neuron — Use EAGLE3 from the offline `vllm.LLM` Python API instead of the server.

Tutorial: Configure disaggregated inference with 1P1D and xPyD on vLLM on Neuron

This topic guides you through configuring disaggregated inference (DI) with vLLM on Neuron using a simple 1P1D (one prefill, one decode) example and scaling up to a general xPyD topology. When you have completed it, you will have a working DI deployment and will understand the control and data flow between prefill and decode servers.

Overview

Disaggregated inference separates the prefill phase (prompt processing) from the decode phase (token generation) across different servers. This separation lets you:

- Scale prefill and decode independently based on your traffic mix.
- Use different parallelism configurations for each phase (for example, TP=8 for prefill, DP4×TP8 with expert parallelism for decode).
- Reduce head-of-line blocking where long prompts delay short decode requests.

The architecture has three components:

1. **Prefill server** — runs the model forward pass over the prompt and produces the KV cache.
2. **Decode server** — pulls the KV cache from the prefill server and generates tokens iteratively.
3. **Proxy server** — routes client requests to prefill, then hands off to decode for token generation.

KV cache transfer between prefill and decode uses NiXL over LIBFABRIC (EFA on AWS). The decode server pulls KV blocks directly from the prefill server's device memory.

Request flow:

1. Client sends request to the proxy server.
2. Proxy forwards the request to a prefill server.
3. Prefill server processes the prompt and produces KV cache.
4. Proxy signals the decode server to begin generation.
5. Decode server pulls KV cache from prefill via NiXL/LIBFABRIC.
6. Decode server generates tokens and streams them back through the proxy to the client.

Before you start

This tutorial assumes that you have experience in the following areas:

- Running vLLM on Neuron on a single instance. See Quickstart: Launch an OpenAI-compatible API server with vLLM on Neuron.
- Familiarity with KV cache and its role in LLM inference.
- Understanding of tensor parallelism and data parallelism concepts.

Prerequisites

- **Neuron instance:** A supported Trainium instance with enough NeuronCores for both prefill and decode (for example, `trn2.48xlarge` with 64 NeuronCores). For multi-node, two or more instances with EFA connectivity.
- **vLLM on Neuron environment:** Installed and verified. See How to set up vLLM on Neuron.
- **NiXL installed:** The NiXL KV transfer library. Install with:

  ```
  pip install nixl
  ```

- **Model access:** Ability to pull the model on each server (for example, via Hugging Face Hub or a shared filesystem).

Prepare your environment

Activate the vLLM virtual environment and set the NiXL side channel to listen on all interfaces:

```
source /opt/aws_neuronx_venv_pytorch_inference_vllm/bin/activate
export VLLM_NIXL_SIDE_CHANNEL_HOST="0.0.0.0"
```

Set your model path:

```
MODEL="meta-llama/Llama-3.1-8B-Instruct"
```

Step 1: Launch the prefill server

In this step, you will start the vLLM server in the `kv_producer` role. This server handles prompt processing and produces the KV cache that the decode server will consume.

```
NEURON_VISIBLE_DEVICES=0-7 \
    VLLM_NIXL_SIDE_CHANNEL_PORT=5559 \
    vllm serve $MODEL \
    --port 8100 \
    --tensor-parallel-size 8 \
    --max-num-seqs 1 \
    --max-model-len 4096 \
    --max-num-batched-tokens 4096 \
    --no-enable-chunked-prefill \
    --no-enable-prefix-caching \
    --additional-config \
        '{"nixl_side_channel_port": 5559, '\
        '"neuron_config": {'\
        '"on_device_sampling_config": {"all_greedy": true}, '\
        '"num_batched_tokens_buckets": [4096], '\
        '"num_seqs_buckets": [1]}}' \
    --kv-transfer-config \
        '{"kv_connector": "NixlConnector", '\
        '"kv_role": "kv_producer", '\
        '"kv_buffer_device": "cuda", '\
        '"kv_connector_extra_config": {'\
        '"backends": ["LIBFABRIC"]}}' \
    --hf-overrides '{"quantization_config": {}}'
```

Key parameters:

- `NEURON_VISIBLE_DEVICES=0-7` — restricts this server to NeuronCores 0–7.
- `VLLM_NIXL_SIDE_CHANNEL_PORT=5559` — the port NiXL uses for metadata exchange between servers.
- `--kv-transfer-config` — configures the NiXL connector with `kv_role: "kv_producer"` (this server produces KV cache).
- `--port 8100` — the prefill server's API port.

Wait for the server to print `Uvicorn running on http://0.0.0.0:8100`.

Step 2: Launch the decode server

In this step, you will start the decode server in the `kv_consumer` role. This server pulls KV cache from the prefill server and generates tokens.

The decode server can use a different parallelism configuration. This example uses DP=4 with TP=8 and expert parallelism on NeuronCores 8–39:

```
NEURON_VISIBLE_DEVICES=8-39 \
    VLLM_NIXL_SIDE_CHANNEL_PORT=5659 \
    vllm serve $MODEL \
    --port 8200 \
    --tensor-parallel-size 8 \
    --data-parallel-size 4 \
    --enable-expert-parallel \
    --max-num-seqs 1 \
    --max-model-len 4096 \
    --max-num-batched-tokens 4096 \
    --no-enable-chunked-prefill \
    --no-enable-prefix-caching \
    --additional-config \
        '{"nixl_side_channel_port": 5659, '\
        '"neuron_config": {'\
        '"on_device_sampling_config": {"all_greedy": true}, '\
        '"num_batched_tokens_buckets": [4096], '\
        '"num_seqs_buckets": [1]}}' \
    --kv-transfer-config \
        '{"kv_connector": "NixlConnector", '\
        '"kv_role": "kv_consumer", '\
        '"kv_buffer_device": "cuda", '\
        '"kv_connector_extra_config": {'\
        '"backends": ["LIBFABRIC"]}}' \
    --hf-overrides '{"quantization_config": {}}'
```

Key parameters:

- `NEURON_VISIBLE_DEVICES=8-39` — uses NeuronCores 8–39 (32 cores for DP4×TP8).
- `VLLM_NIXL_SIDE_CHANNEL_PORT=5659` — a different port than the prefill server to avoid conflicts on the same host.
- `kv_role: "kv_consumer"` — this server consumes KV cache from the producer.
- `--data-parallel-size 4 --enable-expert-parallel` — the decode side can use more parallelism for higher throughput.

Wait for the server to print `Uvicorn running on http://0.0.0.0:8200`.

Note

The prefill and decode servers do not need identical parallelism configurations. You can use TP=8 for prefill (optimized for large prompt processing) and DP4×TP8 for decode (optimized for concurrent token generation). This is called **hybrid TP** and is a key advantage of disaggregated inference.

Step 3: Launch the proxy server

In this step, you will start the proxy server that routes client requests between prefill and decode.

```
python3 examples/vllm_neuron/vllm/disaggregated_inference/toy_proxy_server.py \
    --port 8000 \
    --prefiller-ports 8100 \
    --decoder-ports 8200
```

The proxy listens on port 8000 and coordinates the request lifecycle: it sends prompts to the prefill server, then hands off to the decode server for token generation.

> **Note**
>
> The `toy_proxy_server.py` is included in the vLLM-Neuron repository under `examples/vllm_neuron/vllm/disaggregated_inference/`. For production deployments, use an orchestrator like AIBrix or production-stack instead.

Step 4: Validate the 1P1D deployment

In this step, you will send a request through the proxy and confirm it completes end-to-end.

```
curl -s http://localhost:8000/v1/completions \
    -H "Content-Type: application/json" \
    -d "{
      \"model\": \"$MODEL\",
      \"prompt\": \"Count the number 1, 2, 3\",
      \"max_tokens\": 200
    }"
```

A successful JSON response with generated text confirms:

1. The proxy routed the request to the prefill server (port 8100).
2. The prefill server processed the prompt and produced KV cache.
3. The decode server (port 8200) pulled the KV cache via NiXL over LIBFABRIC.
4. The decode server generated tokens and returned them through the proxy.

Step 5: Scale to xPyD (multi-node)

In this step, you will generalize the topology to multiple prefill and decode nodes across separate instances.

For a multi-node deployment, the configuration changes are:

1. Each server runs on its own instance (no `NEURON_VISIBLE_DEVICES` partitioning needed).
2. The proxy server specifies remote hosts.
3. Security groups must allow traffic between instances on the server ports and NiXL side channel ports.

Prefill server (on instance at `10.0.1.10`):

```
VLLM_NIXL_SIDE_CHANNEL_PORT=5559 \
    vllm serve $MODEL \
    --port 8100 \
    --tensor-parallel-size 8 \
    --max-num-seqs 4 \
    --max-model-len 8192 \
    --max-num-batched-tokens 8192 \
    --no-enable-prefix-caching \
    --additional-config \
        '{"nixl_side_channel_port": 5559, '\
        '"neuron_config": {'\
        '"num_batched_tokens_buckets": [8192], '\
        '"num_seqs_buckets": [4]}}' \
    --kv-transfer-config \
        '{"kv_connector": "NixlConnector", '\
        '"kv_role": "kv_producer", '\
        '"kv_buffer_device": "cuda", '\
        '"kv_connector_extra_config": {'\
        '"backends": ["LIBFABRIC"]}}'
```

Decode server (on instance at `10.0.1.20`):

```
VLLM_NIXL_SIDE_CHANNEL_PORT=5659 \
    vllm serve $MODEL \
    --port 8200 \
    --tensor-parallel-size 8 \
    --max-num-seqs 4 \
    --max-model-len 8192 \
    --max-num-batched-tokens 8192 \
    --no-enable-prefix-caching \
    --additional-config \
        '{"nixl_side_channel_port": 5659, '\
        '"neuron_config": {'\
        '"num_batched_tokens_buckets": [8192], '\
        '"num_seqs_buckets": [4]}}' \
    --kv-transfer-config \
        '{"kv_connector": "NixlConnector", '\
        '"kv_role": "kv_consumer", '\
        '"kv_buffer_device": "cuda", '\
        '"kv_connector_extra_config": {'\
        '"backends": ["LIBFABRIC"]}}'
```

Proxy server with remote hosts:

```
python3 examples/vllm_neuron/vllm/disaggregated_inference/toy_proxy_server.py \
    --port 8000 \
    --prefiller-host 10.0.1.10 --prefiller-port 8100 \
    --decoder-host 10.0.1.20 --decoder-port 8200
```

To scale to **2P3D** (2 prefill, 3 decode), add more hosts:

```
python3 examples/vllm_neuron/vllm/disaggregated_inference/toy_proxy_server.py \
    --port 8000 \
    --prefiller-host 10.0.1.10 10.0.1.11 \
    --prefiller-port 8100 8100 \
    --decoder-host 10.0.1.20 10.0.1.21 10.0.1.22 \
    --decoder-port 8200 8200 8200
```

The proxy round-robins across all prefill and decode instances.

Confirmation

You have a working DI deployment that:

- Separates prefill and decode across different NeuronCore partitions or instances.
- Transfers KV cache via NiXL over LIBFABRIC.
- Routes requests through a proxy server.
- Supports asymmetric parallelism (different TP/DP/EP on prefill vs decode).
- Scales to arbitrary xPyD topologies by adding more servers and updating the proxy.

Common issues

- **Requests stall after prefill completes:** Verify that `VLLM_NIXL_SIDE_CHANNEL_HOST="0.0.0.0"` is set on both servers. Check that the NiXL side channel ports (5559, 5659) are reachable between the servers.
- **``Connection refused`` from proxy to servers:** Both vLLM servers must be fully started before the proxy can route requests. First-time Neuron compilation can take several minutes — wait for the `Uvicorn running` message on each server.
- **KV transfer timeout or errors:** Ensure LIBFABRIC/EFA is available. On multi-node, confirm instances are in the same placement group or subnet. Check that `nixl` is installed (`python -c "import nixl"`).

- **Decode server OOM:** The decode server must hold KV cache for all in-flight requests. Reduce `--max-num-seqs` or `--max-model-len` if memory is tight.
- **Port conflicts on single-node:** When running both servers on one instance, use different `VLLM_NIXL_SIDE_CHANNEL_PORT` values (for example, 5559 for prefill, 5659 for decode) and different `--port` values.

Clean up

Stop all processes:

```
pkill -f "vllm serve"
pkill -f "toy_proxy_server"
```

If you launched EC2 instances specifically for this tutorial, terminate them to avoid ongoing charges.

Next steps

- How to use vLLM on Neuron features in your deployment — Configure prefix caching, speculative decoding, and other features alongside DI.